

AGENT ENGINEER

Offer 100 题

生产级智能体工程面试手册

从 Agent Loop 到 Multi-Agent、Context、RAG、Durable Execution、Eval、Security
与 Production System Design



剑指 Offer 式训练结构 · 原创内容与版式

100 道题 | 10 条能力轴 | 30 秒回答 | 深挖解析 | 追问链 | 易错点 | Know-Why | Know-How

2026 · Agent Engineering Interview Edition

出版说明

原创说明 本手册借鉴《剑指 Offer》“高频问题 + 逐层拆解 + 面试追问 + 易错点”的学习方法，但不复制其原文、题目、插图或版面设计。本文档内容围绕 Agent 工程独立编写。

资料基础 核心知识结构参考用户提供的《AI Agents in Action》《AI Agent 开发与应用：基于大模型的智能体构建》《AI Agent 智能工作流》《字节跳动 RAG 实践手册》。其中 Agent 组件、记忆、工具、规划、评估、多智能体、RAG、监控与安全等术语沿用这些资料的通用技术框架。

工程扩展 Durable Execution、幂等、Saga、Circuit Breaker、Deadline Propagation、Failure Attribution、Multi-Tenant Isolation 等内容，是为“生产级 Agent 面试”补充的系统工程化展开，不应被理解为上述书籍逐字观点。

适合谁

- Agent / LLM Application Engineer 求职者
- 从 Prompt/RAG 开发转向 Production Agent 的工程师
- 需要面试 Agent 候选人的技术负责人
- 希望用体系化方法查缺补漏的开发者

使用方式

- 第一遍：只看“题目 + 30 秒回答”，建立知识骨架。
- 第二遍：遮住答案，按“Detect → Classify → Contain → Recover → Preserve → Verify”完整作答。
- 第三遍：只练标记“必考/压轴”的题，并模拟追问。

前言 | Agent 面试已经变成“可靠性面试”

过去的 Agent 面试常问“用过 LangChain 吗”“ReAct 是什么”。真正进入生产以后，问题迅速变成：多 Agent 通信丢消息怎么办？长上下文爆掉如何恢复？工具超时能不能重试？Agent 跑偏如何中途发现？一次错误动作如何限制影响范围？

这些问题的共同点是：它们并不要求你把模型变成确定性程序，而是要求你在承认模型具有概率性的前提下，建立确定性的工程边界。也就是说，优秀的 Agent Engineer 不只是会让 Agent 成功，更要让它“可控地失败、可解释地失败、可恢复地失败”。

本书把 100 道题按十条能力轴组织。每道题都尽量回答两个问题：Know-Why——为什么这是生产 Agent 的关键问题；Know-How——你在系统里究竟应该落什么机制。面试中，只要能从“概念”继续讲到“失效模式、控制点、指标和恢复路径”，答案就会明显从框架使用者升级到系统工程师。

全书主线 Goal → Plan → Context → Action → Tool → Observation → State → Evaluation → Replan → Stop。每个箭头都追问：失败怎么办？重复怎么办？超时怎么办？跑偏怎么办？怎么知道它跑偏了？怎么恢复？怎么证明修好了？

能力地图 | 10 条轴看懂 Agent 岗到底考什么

能力轴	建议权重	面试官真正想看
Agent Loop / 架构	8%	状态、控制流、Harness、完成判定
Planning / 控制	10%	规划、反思、跑偏检测、终止
Tool / MCP	12%	工具选择、幂等、Gateway、副作用
Multi-Agent	12%	协议、Handoff、隔离、容错
Context / Memory	12%	压缩、重置、长期记忆、冲突
Agentic RAG	10%	触发、检索、Rerank、ACL、评估
Runtime / Fault Tolerance	12%	Checkpoint、Retry、Saga、恢复
Eval / Observability	10%	Trace、归因、回归、SLO
Security / HITL	8%	注入、最小权限、Sandbox、审批
Performance / Cost	6%	Latency、成本、规模化

六步答题框架：Reliability First

遇到任何“Agent 出问题怎么办”的题，优先按以下六步组织答案。

01 Detect 怎么知道出问题：监控、trace、异常信号。	02 Classify 定位 Planner、Context、RAG、Tool、Runtime 或 Security。
03 Contain 限制 blast radius：停止、限步、权限、Circuit Breaker、HITL。	04 Recover Retry、Fallback、Replan、Compensation、Resume、Reassign。
05 Preserve 保留 Checkpoint、Operation ID、Artifact、Execution Log。	06 Verify 用 Replay、Regression Eval、A/B 和线上指标验证。
示范句式 我不会先直接 retry。先从 trace 判断失败层级，再按 error taxonomy 判断 retryability；对有副作用的工具结合 idempotency key 和 operation status；超过预算后 fallback/replan，高风险动作进入 HITL；最后用 replay + regression eval 验证修复。	

目录 | Contents

按能力轴组织，共 10 章 100 题。题号本身就是复习索引。

CHAPTER 01	Agent 架构与 Agent Loop	Q01-Q10
CHAPTER 02	Planning、Reflection 与任务控制	Q11-Q20
CHAPTER 03	Tool Calling、MCP 与外部动作	Q21-Q30
CHAPTER 04	Multi-Agent 通信与协作	Q31-Q40
CHAPTER 05	Context Engineering 与 Memory	Q41-Q50
CHAPTER 06	Agentic RAG	Q51-Q60
CHAPTER 07	Durable Execution 与 Fault Tolerance	Q61-Q70
CHAPTER 08	Evaluation、Tracing 与 Observability	Q71-Q80
CHAPTER 09	Security、Permission 与 HITL	Q81-Q90
CHAPTER 10	性能、成本与综合系统设计	Q91-Q100

CHAPTER 01

Agent 架构与 Agent Loop

先把 Agent 看成一个可恢复的运行时，而不是一次模型调用。

关键词 Agent Loop / State / Harness / Completion

本章训练目标

- 能在白板上画出关键控制边界，而不是只报框架名。
- 能从 failure mode 反推防护机制、状态和指标。
- 面对连续追问时，始终回到“为什么 + 怎么落地”。

01. LLM、Workflow 和 Agent 有什么区别？什么时候不应该使用 Agent？

频率 必考

难度 中

能力轴 Agent 架构

面试官在考什么 Agent 的价值来自运行时适应性；代价也来自运行时不确定性。面试官在看你是否有‘先选控制范式，再选框架’的工程意识。

30 秒回答

LLM 是概率式能力组件，Workflow 是预定义控制流，Agent 则让模型在运行时决定部分控制流。若任务规则稳定、步骤确定、风险高且可枚举，优先 Workflow；只有当路径无法穷举、需要动态工具选择或环境反馈时，Agent 才有收益。

深挖解析

- 用‘控制流由谁决定’区分三者，而不是按产品名称区分。
- 评估自治带来的收益是否覆盖不确定性、成本、延迟和审计复杂度。
- 高风险动作可采用 Workflow 外壳 + Agent 局部决策的混合结构。

连续追问

Q1 固定审批流程为什么通常不该做成 Agent？

Q2 怎样判断一个流程值得 Agent 化？

易错回答 把‘用了 LLM’等同于‘Agent’，或默认 Agent 一定比 Workflow 高级。

Know-Why Agent 的价值来自运行时适应性；代价也来自运行时不确定性。面试官在看你是否有‘先选控制范式，再选框架’的工程意识。

Know-How 先画任务 DAG，标出不可预先确定的节点；只有这些节点交给模型决策，其余用确定性代码约束。

一句话收束 不要只讲“模型怎么想”，要继续讲“运行时如何验证、限制和恢复”。

02. 不使用任何 Agent 框架，如何从零实现一个最小 Agent Loop?

频率 必考

难度 中

能力轴 Agent 架构

面试官在考什么 框架最终都会落到循环与状态迁移。能手写最小 Loop，才能真正理解 LangGraph、OpenAI Agents SDK 等框架在替你处理什么。

30 秒回答

最小闭环是：模型读取目标与状态 -> 产生工具调用或最终答案 -> 执行工具 -> 将 observation 写回状态 -> 再次调用模型，直到满足终止条件。生产版还必须加入 step budget、timeout、schema 校验、权限、trace 和 checkpoint。

深挖解析

- messages 只是状态的一部分，任务状态、工具结果、预算和错误状态应结构化保存。
- 终止不只看模型 stop_reason，还要验证业务完成条件。
- 工具执行与模型调用要以 span 形式可追踪。

连续追问

Q1 stop_reason 谁负责判断？

Q2 tool result 应怎样进入下一轮 context？

易错回答 只写 while tool_call 循环，却没有最大步数、错误处理和完成验证。

Know-Why 框架最终都会落到循环与状态迁移。能手写最小 Loop，才能真正理解 LangGraph、OpenAI Agents SDK 等框架在替你处理什么。

Know-How 定义 AgentState 数据结构和 transition 函数；将 model_step、tool_step、verify_step 解耦，并为每个 transition 写可重放日志。

一句话收束 不要只讲“模型怎么想”，要继续讲“运行时如何验证、限制和恢复”。

03. 为什么说 Agent Loop 本质上可以看成状态机？State 里应该保存什么？

频率 高频

难度 中

能力轴 Agent 架构

面试官在考什么 长任务、暂停恢复、故障重试都依赖显式状态；如果关键事实只存在模型上下文中，就无法可靠恢复。

30 秒回答

Agent 每一步都从当前状态读取信息并产生下一状态，因此可以建模为状态机或有向图。State 至少包含目标、当前计划、已完成步骤、消息上下文、工具结果、预算、错误、权限上下文和 checkpoint 元数据。

深挖解析

- 把不可丢失的业务状态和可压缩的语言上下文分开。
- 状态迁移应尽量显式，便于恢复、回放和测试。
- 对于并发 worker，还需版本号或乐观锁字段。

连续追问

Q1 哪些字段不能只存在 prompt 里？

Q2 State 如何做版本升级？

易错回答 把完整 chat history 当作唯一状态源。

Know-Why 长任务、暂停恢复、故障重试都依赖显式状态；如果关键事实只存在模型上下文中，就无法可靠恢复。

Know-How 将状态分成 control state、business state、context state 三层，并为每层定义持久化与生命周期策略。

一句话收束 不要只讲“模型怎么想”，要继续讲“运行时如何验证、限制和恢复”。

04. 生产 Agent 中，哪些逻辑交给 LLM，哪些逻辑必须写死在代码里？

频率 必考

难度 中

能力轴 Agent 架构

面试官在考什么 LLM 是概率系统，不能保证 invariant 始终成立。生产系统要把‘可能遵守’变成‘无法绕过’。

30 秒回答

把模糊判断、语义理解、开放式规划交给模型；把权限、金额上限、幂等、数据隔离、schema、超时、次数限制等 invariant 写在代码和可信执行层。原则是：模型可以建议，代码必须裁决不可违反的约束。

深挖解析

- 模型适合处理 fuzzy policy，不适合作为安全边界。
- 所有 side-effect tool 在执行前做确定性校验。
- 模型输出用结构化 schema 缩小自由度。

连续追问

Q1 权限判断能不能让 LLM 做？

Q2 业务规则很复杂时如何避免全写 prompt？

易错回答 把关键安全规则写进 system prompt 后就认为足够安全。

Know-Why LLM 是概率系统，不能保证 invariant 始终成立。生产系统要把‘可能遵守’变成‘无法绕过’。

Know-How 建立 policy engine / tool gateway；LLM 输出 action intent，执行层再做 ACL、额度和参数校验。

一句话收束 不要只讲“模型怎么想”，要继续讲“运行时如何验证、限制和恢复”。

05. Graph Engineer 和 Loop Engineer 的核心区别是什么？

频率 高频

难度 中

能力轴 Agent 架构

面试官在考什么 这实际上是‘自由度放在哪一层’的问题。面试官关注你如何在自治与控制之间做边界设计。

30 秒回答

Loop Engineer 关注通用循环和动态自治，Graph Engineer 把关键状态、分支、暂停点和恢复点显式建模。复杂生产任务通常用 graph 约束骨架、在节点内部保留 agentic loop。

深挖解析

- Graph 提高可观测性和可恢复性，但降低自由度。
- 纯 Loop 适合探索性任务；强业务流程更适合显式图。
- 最佳实践往往是 deterministic shell + agentic core。

连续追问

Q1 什么时候 graph 会过度设计？

Q2 如何从自由 Loop 迁移到显式 Graph？

易错回答 把 Graph 和 Loop 当成互斥技术选型。

Know-Why 这实际上是‘自由度放在哪一层’的问题。面试官关注你如何在自治与控制之间做边界设计。

Know-How 先从 trace 中识别稳定重复的阶段，把它们固化成 graph 节点；把真正不可枚举的步骤保留为 loop。

一句话收束 不要只讲“模型怎么想”，要继续讲“运行时如何验证、限制和恢复”。

06. Agent 的最终完成条件如何定义？为什么不能相信模型说‘完成了’？

频率 高频

难度 中

能力轴 Agent 架构

面试官在考什么 Agent 最危险的幻觉之一是‘完成幻觉’：它认为动作成功，但外部世界并未改变。

30 秒回答

模型的‘done’只是声明，不是证据。完成条件应由外部可验证事实定义，例如目标文件存在且测试通过、工单状态已更新、订单已创建且 ID 可查询。最终由 verifier 或业务 API 验证。

深挖解析

- 区分 transport complete、agent complete、business complete。
- 为任务定义 machine-checkable acceptance criteria。
- 无法完全自动验证时再引入 HITL。

连续追问

Q1 创意任务没有确定答案怎么验证？

Q2 验证器本身是 LLM 怎么办？

易错回答 仅看自然语言里的‘任务已完成’。

Know-Why Agent 最危险的幻觉之一是‘完成幻觉’：它认为动作成功，但外部世界并未改变。

Know-How 在任务创建时同步生成 acceptance criteria；结束前执行 verify_step，失败则 replan 或升级人工。

一句话收束 不要只讲“模型怎么想”，要继续讲“运行时如何验证、限制和恢复”。

07. Agent 的 Structured Output 校验失败怎么办？

频率 高频

难度 中

能力轴 Agent 架构

面试官在考什么 结构化输出是 Agent 与程序世界的契约；契约失败不应直接污染后续状态。

30 秒回答

先在边界层做 schema validation；对可修复错误进行有限次 repair/retry，对语义不合法的字段重新询问模型；超过预算则 fallback 或请求用户补充。不要用脆弱正则硬修所有异常。

深挖解析

- 区分 syntax error 与 semantic error。
- 优先使用受约束输出或原生 schema。
- 重试提示中反馈具体 validation error，而不是只说‘格式错误’。

连续追问

Q1 连续三次 schema 都失败怎么办？

Q2 parser repair 会不会掩盖模型问题？

易错回答 无限 retry，或者静默丢字段继续执行。

Know-Why 结构化输出是 Agent 与程序世界的契约；契约失败不应直接污染后续状态。

Know-How 用 Pydantic/JSON Schema 做边界校验，错误分类后决定 retry、repair、fallback 或 fail-fast。

一句话收束 不要只讲“模型怎么想”，要继续讲“运行时如何验证、限制和恢复”。

08. 一个 Agent 系统应该有几层状态？

频率 中频

难度 中

能力轴 Agent 架构

面试官在考什么 状态分层是故障恢复、隐私治理和上下文压缩的基础。

30 秒回答

通常至少区分 Run State、Session State、User/Memory State 和 Durable Business State。它们的生命周期、访问权限和一致性要求不同，不能统一塞进 message history。

深挖解析

- Run State 随一次执行结束；Session State 跨多轮对话；User State 跨会话；Business State 属于外部系统真相。
- 不同层使用不同存储和 TTL。
- 敏感状态需要租户隔离和访问审计。

连续追问

Q1 Memory 属于 Session 还是 User State？

Q2 哪些状态可以最终一致？

易错回答 用一个 Redis key 存所有状态，不区分生命周期和权威性。

Know-Why 状态分层是故障恢复、隐私治理和上下文压缩的基础。

Know-How 为每个字段写 owner、source of truth、TTL、consistency、privacy 五个属性，再决定存储位置。

一句话收束 不要只讲“模型怎么想”，要继续讲“运行时如何验证、限制和恢复”。

09. 什么是 Agent Harness? 它和 Prompt、Agent Framework 的区别是什么?

频率 必考

难度 难

能力轴 Agent 架构

面试官在考什么 复杂 Agent 的性能取决于模型和环境共同作用。好的 Harness 把模型的概率能力约束成可操作、可恢复的系统行为。

30 秒回答

Prompt 是模型输入策略，Framework 提供编排 API，Harness 则是让 Agent 稳定工作的完整运行环境：工具、上下文、权限、状态、checkpoint、eval、trace、预算和恢复机制都属于 Harness。

深挖解析

- Harness 是‘模型周围的工程系统’，不是单一库。
- 同一模型换 Harness，成功率可能显著变化。
- 生产成熟度通常体现在 Harness，而不是 prompt 技巧数量。

连续追问

Q1 为什么换模型不一定解决 Agent 稳定性问题?

Q2 Harness 哪些能力应平台化?

易错回答 把 Harness 仅理解为 system prompt 或工具集合。

Know-Why 复杂 Agent 的性能取决于模型和环境共同作用。好的 Harness 把模型的概率能力约束成可操作、可恢复的系统行为。

Know-How 建立统一 runtime 层：context builder、tool gateway、state store、policy、trace、eval hooks。

一句话收束 不要只讲“模型怎么想”，要继续讲“运行时如何验证、限制和恢复”。

10. 请构建一个 Agent Failure Taxonomy。

频率 必考

难度 难

能力轴 Agent 架构

面试官在考什么 没有分类就无法统计、归因、设置不同恢复策略，也无法判断优化模型、RAG 还是工具哪一层更有效。

30 秒回答

按故障发生层分类比按报错文本分类更有效：Goal/Instruction、Planning、Context/Memory、Retrieval、Model、Tool、State、Multi-Agent Coordination、Infrastructure、Security、Business Rule。每类再标注 retryable、recoverable、severity 和 owner。

深挖解析

- Failure taxonomy 是自动恢复和故障归因的共同语言。
- 同一个最终错误可能来自不同上游原因。
- 要记录根因与传播路径，而非只记录最后 exception。

连续追问

Q1 如何把 taxonomy 接入线上监控？

Q2 哪些错误需要自动升级人工？

易错回答 把所有失败都归成 ‘LLM 不稳定’ 。

Know-Why 没有分类就无法统计、归因、设置不同恢复策略，也无法判断优化模型、RAG 还是工具哪一层更有效。

Know-How 给 trace span 打 failure_type/error_code；周报按层聚合 task failure rate，并为每类绑定 runbook。

一句话收束 不要只讲“模型怎么想”，要继续讲“运行时如何验证、限制和恢复”。

CHAPTER 02

Planning、Reflection 与任务控制

真正的难点不是会规划，而是能发现跑偏并控制重规划成本。

关键词 Plan / ReAct / Reflection / Drift / Stop

本章训练目标

- 能在白板上画出关键控制边界，而不是只报框架名。
- 能从 failure mode 反推防护机制、状态和指标。
- 面对连续追问时，始终回到“为什么 + 怎么落地”。

11. ReAct、Plan-and-Execute、Reflection 分别适合什么任务？

频率 高频

难度 中

能力轴 Planning、Reflection

面试官在考什么 规划范式选择的本质是用多少前瞻换多少运行时灵活性。

30 秒回答

ReAct 适合环境反馈频繁、路径不确定的任务；Plan-and-Execute 适合依赖关系清晰、步骤较长的任务；Reflection 适合存在可评估结果、允许迭代改进的任务。生产系统经常组合使用，而不是三选一。

深挖解析

- ReAct 的优势是即时适应，缺点是局部贪心与循环。
- Plan-and-Execute 降低每步规划成本，但初始计划可能过时。
- Reflection 只有在反馈信号有信息量时才值得付额外 token。

连续追问

Q1 什么时候 Reflection 反而降低效果？

Q2 计划多久重算一次？

易错回答 背三种模式定义，却不讨论任务特征和成本。

Know-Why 规划范式选择的本质是用多少前瞻换多少运行时灵活性。

Know-How 用任务长度、环境变化率、可验证性、工具延迟四个维度选择策略，并通过 eval 验证。

一句话收束 不要只讲“模型怎么想”，要继续讲“运行时如何验证、限制和恢复”。

12. Planner 输出的计划为什么不能直接执行？

频率 必考

难度 中

能力轴 Planning、Reflection

面试官在考什么 LLM 可能生成不存在的工具、无效顺序或越权动作。计划必须从语言建议转成受约束的执行图。

30 秒回答

Planner 只是提出候选控制流。执行前要做 schema、依赖、工具存在性、权限、预算、可行性和安全检查；涉及副作用的步骤还要经过 policy/HITL。

深挖解析

- 计划可以是模型生成，但计划约束应由程序验证。
- 对依赖图做 cycle / missing dependency 检查。
- 对资源和 deadline 做 feasibility check。

连续追问

Q1 如何验证自然语言计划？

Q2 计划校验失败后是修复还是重规划？

易错回答 把 planner 当作绝对可信的调度器。

Know-Why LLM 可能生成不存在的工具、无效顺序或越权动作。计划必须从语言建议转成受约束的执行图。

Know-How 让 planner 输出结构化 TaskGraph；编译阶段做静态检查，执行阶段做动态 guard。

一句话收束 不要只讲“模型怎么想”，要继续讲“运行时如何验证、限制和恢复”。

13. Agent 执行一半跑偏了，如何在中间发现？

频率 必考

难度 难

能力轴 Planning、Reflection

面试官在考什么 Agent 错误会沿后续步骤放大。越早检测，blast radius 越小，恢复成本越低。

30 秒回答

不要等最终答案。为任务定义 goal invariant、milestone 和 progress signal；每 N 步或关键动作后比较当前状态与目标，检测重复动作、信息增益下降、约束违反和计划偏移，并触发 replan/stop/HITL。

深挖解析

- Drift detection 既有规则信号，也可有 evaluator。
- trajectory-level 评估比只看 final answer 更早暴露问题。
- 高风险动作前必须强制 precondition check。

连续追问

Q1 怎么定义 progress？

Q2 Evaluator 也可能判断错怎么办？

易错回答 只靠 max_steps 防跑偏。

Know-Why Agent 错误会沿后续步骤放大。越早检测，blast radius 越小，恢复成本越低。

Know-How 记录 expected_next_state 与 actual_state；配置 repeated-call detector、constraint checker 和 milestone verifier。

一句话收束 不要只讲“模型怎么想”，要继续讲“运行时如何验证、限制和恢复”。

14. 计划执行到第三步发现环境变化，全部重规划还是局部重规划？

频率 中频

难度 难

能力轴 Planning、Reflection

面试官在考什么 长任务的成本常常不在执行，而在错误重算。局部重规划需要你显式理解依赖关系。

30 秒回答

看变化影响的依赖范围。若只影响局部子图，保留已验证结果并局部重规划；若目标、关键假设或上游约束失效，则全局重规划。原则是最小化无效重算，同时避免沿用已失效假设。

深挖解析

- 需要依赖图和 assumption tracking。
- 已完成步骤要区分可复用 artifact 与环境相关结果。
- 重规划前先判断哪些状态仍然有效。

连续追问

Q1 怎样判断一个结果是否 still valid?

Q2 环境频繁变化怎么办？

易错回答 每次变化都从零开始，或完全不重规划。

Know-Why 长任务的成本常常不在执行，而在错误重算。局部重规划需要你显式理解依赖关系。

Know-How 为 task/result 保存依赖、版本与 freshness；失效时做 downstream invalidation。

一句话收束 不要只讲“模型怎么想”，要继续讲“运行时如何验证、限制和恢复”。

15. ReAct Agent 为什么经常死循环？怎么识别？

频率 必考

难度 中

能力轴 Planning、Reflection

面试官在考什么 死循环是 Agent 自治的典型失效模式；识别循环模式比硬限步更能保持成功率。

30 秒回答

常见原因是 observation 没有带来新信息、工具失败被模型误判、目标模糊或模型在局部策略间震荡。检测信号包括相同工具+参数重复、状态哈希重复、计划边循环、信息增益接近零。

深挖解析

- step limit 只能兜底，不能解释根因。
- 区分 retry loop、tool-selection loop、reasoning loop。
- 循环可用 trajectory signature 检测。

连续追问

Q1 什么时候重复调用其实是合理的？

Q2 如何避免误杀长轮询任务？

易错回答 只把 max_steps 调小。

Know-Why 死循环是 Agent 自治的典型失效模式；识别循环模式比硬限步更能保持成功率。

Know-How 定义 normalized action signature；连续重复超过阈值且 observation 无变化时进入 replan 或 fallback。

一句话收束 不要只讲“模型怎么想”，要继续讲“运行时如何验证、限制和恢复”。

16. max_steps 设置 10 还是 100? 依据是什么?

频率 中频

难度 中

能力轴 Planning、Reflection

面试官在考什么 步数上限同时控制 runaway 成本和错误传播范围。

30 秒回答

它是风险和成本预算，不应凭感觉。依据任务 DAG 深度、历史成功轨迹的 step 分布、token/latency budget、工具成本和 SLO 设置，并可按任务类型动态分配。

深挖解析

- 分析 P50/P95 successful steps。
- 给异常轨迹留少量恢复余量。
- 对昂贵或高风险工具设置独立调用上限。

连续追问

Q1 动态 step budget 怎么做？

Q2 为什么不是越大成功率越高？

易错回答 统一写死 20，或为了提高成功率无限放大。

Know-Why 步数上限同时控制 runaway 成本和错误传播范围。

Know-How 从离线 trace 统计成功任务步数分布，按任务复杂度 bucket 配额，并在线监控 budget exhaustion rate。

一句话收束 不要只讲“模型怎么想”，要继续讲“运行时如何验证、限制和恢复”。

17. 什么时候 Agent 应该向用户澄清，而不是继续猜？

频率 高频

难度 中

能力轴 Planning、Reflection

面试官在考什么 澄清是一种成本控制策略，不只是对话礼貌。

30 秒回答

当关键参数缺失且错误代价高、多个解释会导致不同副作用、或继续探索成本高于一次澄清时应询问。可以用 $\text{uncertainty} \times \text{impact} \times \text{irreversibility}$ 作为澄清阈值。

深挖解析

- 低风险信息型任务可先做合理假设并声明。
- 高风险写操作缺关键字段时必须阻断。
- 澄清问题应最小化，一次询问解决最大信息缺口。

连续追问

Q1 怎么避免 Agent 过度提问？

Q2 如何量化 uncertainty？

易错回答 任何不确定都问，导致体验差；或什么都不问，直接猜。

Know-Why 澄清是一种成本控制策略，不只是对话礼貌。

Know-How 把关键字段标注 required/optional/risk；对 required+high-impact 缺失直接 ask，其他按 confidence policy 处理。

一句话收束 不要只讲“模型怎么想”，要继续讲“运行时如何验证、限制和恢复”。

18. Reflection 为什么不是简单让模型‘再想一次’？

频率 高频

难度 中

能力轴 Planning、Reflection

面试官在考什么 没有外部约束的自我纠错容易共享同一盲点。

30 秒回答

有效 Reflection 需要新的反馈信号：测试结果、外部证据、独立 rubric、不同模型或 critic。若只是同一上下文里重复生成，往往只是增加 token，未必增加独立信息。

深挖解析

- Reflection 的价值来自 error signal。
- 可用 verifier/critic 与 actor 解耦。
- 反思次数要有边际收益阈值。

连续追问

Q1 什么时候 self-reflection 有效？

Q2 Actor 和 Judge 用同一模型有什么风险？

易错回答 把‘请再认真想想’当成可靠评估机制。

Know-Why 没有外部约束的自我纠错容易共享同一盲点。

Know-How 优先接入测试、schema、检索证据和独立 judge；只有检测到可修复问题才触发 reflection。

一句话收束 不要只讲“模型怎么想”，要继续讲“运行时如何验证、限制和恢复”。

19. 一个需要运行 2 小时的任务如何拆分，避免后期越来越跑偏？

频率 必考

难度 难

能力轴 Planning、Reflection

面试官在考什么 随着轨迹变长，context pollution、状态遗失和错误传播都会增加。分段能压缩 blast radius。

30 秒回答

把任务拆成可独立验证的 milestone，每阶段输出结构化 artifact 并 checkpoint；阶段间通过最小 handoff context 继续，而不是无限累积对话历史。关键节点做 verifier，失败只回滚局部阶段。

深挖解析

- 长任务需要显式阶段边界。
- artifact 比 narrative summary 更可靠。
- 阶段完成要有 acceptance criteria。

连续追问

Q1 怎样决定阶段粒度？

Q2 什么时候做 context reset？

易错回答 让一个 session 持续运行两小时并依赖模型记住全部历史。

Know-Why 随着轨迹变长，context pollution、状态遗失和错误传播都会增加。分段能压缩 blast radius。

Know-How 用 DAG 划 milestone；每个 milestone 保存 input/output/assumption/test；下一阶段从 artifact + concise context 启动。

一句话收束 不要只讲“模型怎么想”，要继续讲“运行时如何验证、限制和恢复”。

20. 用户在 Agent 执行第 18 步时点击取消，系统如何正确停止？

频率 高频

难度 难

能力轴 Planning、Reflection

面试官在考什么 真正的工作可能已经在外部系统继续执行；错误取消会制造幽灵任务和重复副作用。

30 秒回答

取消必须传播到当前 run、正在执行的工具、子 Agent 和队列任务；对不可中断的 side effect 标记 pending 并查询最终状态。取消后写 checkpoint 和 cleanup 结果，确保再次启动不会重复操作。

深挖解析

- 区分 cooperative cancellation 与 hard kill。
- 工具层最好支持 cancellation token/deadline。
- 取消是状态迁移，不是简单杀进程。

连续追问

Q1 支付请求已经发出去还能取消吗？

Q2 子 Agent 收不到取消信号怎么办？

易错回答 只关闭前端连接或 kill 主进程。

Know-Why 真正的工作可能已经在外部系统继续执行；错误取消会制造幽灵任务和重复副作用。

Know-How run_id 绑定 cancellation token；所有 worker/工具在安全点检查；对外部动作使用 operation_id 做后续 reconciliation。

一句话收束 不要只讲“模型怎么想”，要继续讲“运行时如何验证、限制和恢复”。

CHAPTER 03

Tool Calling、MCP 与外部动作

工具让模型从‘会说’变成‘会做’，也把一致性、权限和副作用带进系统。

关键词 Tool / MCP / Idempotency / Gateway

本章训练目标

- 能在白板上画出关键控制边界，而不是只报框架名。
- 能从 failure mode 反推防护机制、状态和指标。
- 面对连续追问时，始终回到“为什么 + 怎么落地”。

21. Function Calling 背后模型如何决定调用哪个工具？

频率 必考

难度 中

能力轴 Tool Calling、MCP

面试官在考什么 工具选择是模型推理的一部分；设计 action space 直接影响 Agent 行为。

30 秒回答

模型基于当前上下文与工具描述预测最合适的 action/schema；工具名、描述、参数约束和示例都参与决策。工程上应把工具定义看作模型的 action space，而不是普通 API 文档。

深挖解析

- 工具描述要同时写能力边界和不适用条件。
- 参数 schema 越明确，调用稳定性通常越高。
- 可通过减少可见工具降低选择熵。

连续追问

Q1 为什么 tool description 会显著影响准确率？

Q2 如何评估工具选择准确率？

易错回答 认为工具路由完全由框架决定。

Know-Why 工具选择是模型推理的一部分；设计 action space 直接影响 Agent 行为。

Know-How 建立 tool-selection eval，统计 confusion matrix；迭代名称、description、examples 和 router。

一句话收束 工具调用的核心不是 API，而是副作用、一致性与可信执行边界。

22. Agent 总在两个类似工具之间选错，怎么解决？

频率 必考

难度 中

能力轴 Tool Calling、MCP

面试官在考什么 模型在高相似 action space 中会增加选择熵，根因往往是工具设计而非模型智力。

30 秒回答

先减少语义重叠：重新命名、明确适用/禁用条件、补正反例；必要时增加 deterministic router 或按任务动态加载工具。不要只反复改 system prompt。

深挖解析

- 分析错误对的 confusion matrix。
- 对高频混淆工具做职责拆分。
- 工具数量过多时做 capability discovery。

连续追问

Q1 如果业务上两个工具确实很像怎么办？

Q2 Router 用规则还是模型？

易错回答 在提示词里加一句‘请选择正确工具’。

Know-Why 模型在高相似 action space 中会增加选择熵，根因往往是工具设计而非模型智力。

Know-How 用离线样本跑 tool routing eval；按错误类型调整 schema、可见性和 deterministic gating。

一句话收束 工具调用的核心不是 API，而是副作用、一致性与可信执行边界。

23. 工具调用 timeout 了，你会直接 retry 吗？

频率 必考

难度 难

能力轴 Tool Calling、MCP

面试官在考什么 网络超时最危险的地方是结果不确定，盲重试会重复产生副作用。

30 秒回答

不能先默认 retry。先判断操作是否幂等、外部系统是否可能已经执行、错误是否 transient。读操作通常可安全重试；写操作先用 operation/idempotency key 查询状态，再决定重试。

深挖解析

- timeout 表示 ‘不知道结果’，不等于 ‘没有执行’。
- 重试策略应由 tool metadata 标注 retryability。
- 使用 exponential backoff + jitter，并受 deadline 限制。

连续追问

Q1 创建订单 timeout 了怎么办？

Q2 什么时候 fail-fast？

易错回答 看到 timeout 就统一重试三次。

Know-Why 网络超时最危险的地方是结果不确定，盲重试会重复产生副作用。

Know-How 每个 tool 声明 idempotent/side_effect/retry_policy；写操作使用 request_id，并提供 status API。

一句话收束 工具调用的核心不是 API，而是副作用、一致性与可信执行边界。

24. 支付 API 成功但响应丢失，Agent 认为失败并重试，如何避免重复执行？

频率 必考

难度 难

能力轴 Tool Calling、MCP

面试官在考什么 这是分布式系统的 exactly-once 幻觉问题；实际通常用 at-least-once + idempotency 达成业务上的 once。

30 秒回答

客户端生成稳定 idempotency key / operation_id，服务端用它去重；超时后先查询该 operation 状态，再决定补发。Agent checkpoint 中必须保存 operation_id，恢复后沿用同一个 ID。

深挖解析

- 幂等键必须跨 retry 和 crash 稳定。
- 服务端需要持久化去重结果。
- 对不可幂等外部系统使用 reconciliation/compensation。

连续追问

Q1 如果第三方 API 不支持幂等键怎么办？

Q2 幂等记录保留多久？

易错回答 只在客户端记录‘我已经调用过’，却没有服务端去重依据。

Know-Why 这是分布式系统的 exactly-once 幻觉问题；实际通常用 at-least-once + idempotency 达成业务上的 once。

Know-How 在 Tool Gateway 生成 operation_id，持久化 request/result 状态；恢复和重试统一走同一 operation。

一句话收束 工具调用的核心不是 API，而是副作用、一致性与可信执行边界。

25. Tool 返回 partial success，接口应该怎么表达？

频率 中频

难度 中

能力轴 Tool Calling、MCP

面试官在考什么 Agent 的恢复能力受工具契约限制。工具越结构化，越容易做精确恢复。

30 秒回答

不要只有 success=true/false。返回整体 status、成功项、失败项、error_code、retryable、next_action 和 operation_id，让 Agent 能针对失败部分恢复，而不是重复全部动作。

深挖解析

- 批量操作尤其需要 item-level result。
- partial success 要明确是否可安全重试。
- 结果 schema 应便于机器解析，而不是长自然语言。

连续追问

Q1 如何处理 100 个子任务里失败 3 个？

Q2 Agent 怎样决定继续还是回滚？

易错回答 失败时只返回一段错误文本。

Know-Why Agent 的恢复能力受工具契约限制。工具越结构化，越容易做精确恢复。

Know-How 为 tool result 定义统一 envelope: status/data/errors/retry_after/operation_id。

一句话收束 工具调用的核心不是 API，而是副作用、一致性与可信执行边界。

26. 五个工具可以并行调用，如何决定并行还是串行？

频率 高频

难度 中

能力轴 Tool Calling、MCP

面试官在考什么 Agent 的并行性本质上仍受经典并发控制约束。

30 秒回答

先看数据依赖和副作用；无依赖、只读、资源互不冲突的调用适合并行。有共享写状态、顺序语义、限流或高成本时要串行或限并发。

深挖解析

- 并行优化 latency，但会提高资源峰值和结果合并复杂度。
- 对相同 downstream 设置 concurrency limit。
- 并行结果要有稳定 correlation id。

连续追问

Q1 如何避免并行调用互相覆盖？

Q2 最大并发数怎么设？

易错回答 只要能并行就全部 `asyncio.gather`。

Know-Why Agent 的并行性本质上仍受经典并发控制约束。

Know-How 构建 dependency DAG；标注 read/write set 和 rate limit；调度器按资源冲突决定并行度。

一句话收束 工具调用的核心不是 API，而是副作用、一致性与可信执行边界。

27. Tool 返回 100K tokens，直接塞回 context 会发生什么？

频率 必考

难度 中

能力轴 Tool Calling、MCP

面试官在考什么 Context 是稀缺的工作内存，不是持久存储。大工具结果是典型 context pollution 来源。

30 秒回答

会挤占关键上下文、提高延迟与成本，并可能让模型被噪声淹没。应把原始结果外置存储，先结构化过滤/摘要，只把当前决策需要的片段放入 context，同时保留引用以便后续按需检索。

深挖解析

- 区分 raw artifact 与 context view。
- 对大结果做 pagination/search API。
- 摘要不能替代需要精确保真的关键字段。

连续追问

Q1 如何避免摘要丢失关键细节？

Q2 什么时候让 Agent 自己搜索 tool result？

易错回答 把模型 context window 当数据库。

Know-Why Context 是稀缺的工作内存，不是持久存储。大工具结果是典型 context pollution 来源。

Know-How Tool Gateway 将大响应写 artifact store，返回摘要+handle；Agent 用 handle 再检索。

一句话收束 工具调用的核心不是 API，而是副作用、一致性与可信执行边界。

28. MCP 和普通 Function Calling 有什么本质区别？

频率 必考

难度 中

能力轴 Tool Calling、MCP

面试官在考什么 当工具数量和提供方增加时，标准化的发现与交互协议能显著降低集成成本。

30 秒回答

Function Calling 解决‘模型如何表达一个函数调用’，MCP 更偏向标准化模型/Agent 与外部能力之间的协议，包括能力发现、工具/资源暴露、连接与授权语义。它解决的是生态互操作，不只是 JSON schema。

深挖解析

- MCP 把工具接入从单应用私有适配推进到协议化。
- 协议化后仍需要本地权限和风险控制。
- MCP server 不等于可信 server。

连续追问

Q1 MCP 是否替代 Agent framework？

Q2 MCP 如何做权限隔离？

易错回答 把 MCP 简化成‘另一种 function call 格式’。

Know-Why 当工具数量和提供方增加时，标准化的发现与交互协议能显著降低集成成本。

Know-How 在架构上把 MCP client/server 与 Tool Gateway 分层：MCP 负责协议接入，Gateway 负责 policy、审计、限流。

一句话收束 工具调用的核心不是 API，而是副作用、一致性与可信执行边界。

29. 几百个 MCP Tool 全部暴露给模型为什么是坏设计？

频率 高频

难度 难

能力轴 Tool Calling、MCP

面试官在考什么 Action space 越大，决策难度和攻击面都越大。

30 秒回答

工具过多会增加选择熵、prompt 占用、误调用和权限暴露。应该先做 capability discovery / semantic routing，再只加载当前任务最相关、当前用户有权使用的一小组工具。

深挖解析

- 动态工具加载同时提升准确率和安全性。
- 可分 namespace、domain、risk tier。
- 工具搜索本身也需要 eval。

连续追问

Q1 工具搜索选错怎么办？

Q2 多少个可见工具比较合适？

易错回答 认为模型足够强，工具越多越好。

Know-Why Action space 越大，决策难度和攻击面都越大。

Know-How 做两阶段路由：task -> capability set -> concrete tool；每阶段记录 top-k 与选择置信度。

一句话收束 工具调用的核心不是 API，而是副作用、一致性与可信执行边界。

30. 设计一个 Production Tool Gateway。

频率 必考

难度 难

能力轴 Tool Calling、MCP

面试官在考什么 把横切控制集中在 Gateway，能避免每个 Agent 重复实现且行为不一致。

30 秒回答

典型链路是 AuthN/AuthZ -> schema validation -> policy/ACL -> quota/rate limit -> execution -> timeout/retry/idempotency -> result normalization -> audit/trace。对高风险动作还要接 HITL 和 approval token。

深挖解析

- Gateway 是 Agent 与真实副作用之间的可信边界。
- 统一工具错误码、retryability 和 side-effect metadata。
- 对 secret、tenant 和网络出口做隔离。

连续追问

Q1 Gateway 是否会成为单点故障？

Q2 如何支持第三方 MCP server？

易错回答 只做一个 HTTP proxy。

Know-Why 把横切控制集中在 Gateway，能避免每个 Agent 重复实现且行为不一致。

Know-How 按插件注册工具；统一 envelope、policy hooks、idempotency store、trace headers，并做 HA/限流。

一句话收束 工具调用的核心不是 API，而是副作用、一致性与可信执行边界。

CHAPTER 04

Multi-Agent 通信与协作

多 Agent 本质上是带不确定决策器的分布式系统。

关键词 Protocol / Handoff / Isolation / Coordination

本章训练目标

- 能在白板上画出关键控制边界，而不是只报框架名。
- 能从 failure mode 反推防护机制、状态和指标。
- 面对连续追问时，始终回到“为什么 + 怎么落地”。

31. 什么时候 Multi-Agent 比 Single-Agent 更好？什么时候反而更差？

频率 必考

难度 中

能力轴 Multi-Agent 通信

面试官在考什么 多 Agent 是架构复杂度换能力，不是免费提升智力。

30 秒回答

当任务可并行、需要专业角色、需要上下文隔离或独立评审时，多 Agent 有价值；若任务强依赖、规模小、通信成本高或共享状态复杂，单 Agent 往往更稳更便宜。

深挖解析

- 收益来自 specialization、parallelism、independent context。
- 成本来自 coordination、handoff loss、重复 token、错误传播。
- 先证明单 Agent 瓶颈，再引入多 Agent。

连续追问

Q1 如何用 eval 证明 Multi-Agent 有收益？

Q2 多 Agent 会不会只是 prompt role-play？

易错回答 把多 Agent 当作默认升级路线。

Know-Why 多 Agent 是架构复杂度换能力，不是免费提升智力。

Know-How 对同一任务做 single vs multi 的 task success/latency/cost 对照实验，再决定。

一句话收束 多 Agent 的核心不是角色数量，而是协议、状态、隔离和故障域。

32. Orchestrator-Worker 架构怎么设计？

频率 必考

难度 难

能力轴 Multi-Agent 通信

面试官在考什么 真正的多 Agent 系统需要显式控制面，否则会变成不可观测的对话网络。

30 秒回答

Orchestrator 负责目标分解、任务依赖、分配、预算、汇总和完成验证；Worker 只拿最小上下文执行明确子任务。关键是让 orchestrator 管控制面，worker 管数据面，避免所有 Agent 都能随意调度别人。

深挖解析

- 任务应有 task_id、acceptance criteria、deadline、budget。
- Worker 输出结构化 artifact 而非只返回聊天。
- Orchestrator 必须处理失败、超时和重分配。

连续追问

Q1 Orchestrator 自己跑偏怎么办？

Q2 Worker 能否创建新的 Worker？

易错回答 只定义几个角色名，靠群聊让它们自己协作。

Know-Why 真正的多 Agent 系统需要显式控制面，否则会变成不可观测的对话网络。

Know-How 用 TaskGraph + queue；orchestrator 维护 task state machine，worker 无权直接修改全局状态。

一句话收束 多 Agent 的核心不是角色数量，而是协议、状态、隔离和故障域。

33. 多个 Agent 使用异步 JSON 消息通信，你会设计哪些字段？

频率 必考

难度 难

能力轴 Multi-Agent 通信

面试官在考什么 一旦异步化，问题就从‘聊天’变成消息系统的一致性与可追踪性。

30 秒回答

至少包含 message_id、task_id、parent_task_id、sender、receiver、type、payload、timestamp、deadline、attempt、correlation_id、schema_version。若需要可靠传递，还要有 ack/status 与幂等语义。

深挖解析

- 消息协议首先解决身份、关联、顺序和重试。
- payload 要版本化。
- 业务完成信号不要与消息送达 ack 混在一起。

连续追问

Q1 message_id 和 task_id 有什么区别？

Q2 怎么支持 schema 演进？

易错回答 只传 sender、receiver、content 三个字段。

Know-Why 一旦异步化，问题就从‘聊天’变成消息系统的一致性与可追踪性。

Know-How 定义 envelope + typed payload；所有处理器按 message_id 幂等，并将 correlation_id 贯穿 trace。

一句话收束 多 Agent 的核心不是角色数量，而是协议、状态、隔离和故障域。

34. Agent 消息丢失、重复、乱序分别怎么办？

频率 必考

难度 难

能力轴 Multi-Agent 通信

面试官在考什么 多 Agent 通信继承了分布式消息系统的所有经典问题。

30 秒回答

丢失用持久队列+ACK+重试；重复用 message_id/operation_id 去重；乱序用 sequence/version 或按业务 key 串行化。不要追求不存在的完美 exactly-once，而要设计业务幂等。

深挖解析

- 至少一次投递是常见现实。
- 顺序要求通常按 task/resource key 局部保证。
- 重试要有 dead-letter queue。

连续追问

Q1 重复执行无法幂等怎么办？

Q2 如何做 poison message 处理？

易错回答 假设消息总能可靠、按序且只送达一次。

Know-Why 多 Agent 通信继承了分布式消息系统的所有经典问题。

Know-How 使用 durable queue；consumer 先查 dedup store，再执行业务；失败超过阈值进入 DLQ。

一句话收束 多 Agent 的核心不是角色数量，而是协议、状态、隔离和故障域。

35. 主 Agent 怎么知道子 Agent 真正完成？

频率 必考

难度 难

能力轴 Multi-Agent 通信

面试官在考什么 ‘完成’ 是业务语义，不是 transport 语义。

30 秒回答

需要区分消息已送达、Worker 进程结束、子任务完成和业务目标完成。Worker 返回 status+artifact+evidence；Orchestrator 再依据 acceptance criteria 验证，不能只相信字符串 done。

深挖解析

- 完成协议要结构化。
- 对子任务产物做 verifier。
- 必要时查询外部系统确认 side effect。

连续追问

Q1 创意任务如何验证完成？

Q2 Worker 失联但外部动作已成功怎么办？

易错回答 收到 final message 就当完成。

Know-Why ‘完成’ 是业务语义，不是 transport 语义。

Know-How TaskState 采用 QUEUED/RUNNING/VERIFYING/SUCCEEDED/FAILED；成功必须由 verifier 迁移。

一句话收束 多 Agent 的核心不是角色数量，而是协议、状态、隔离和故障域。

36. Handoff 时传整个聊天记录还是最小上下文？

频率 高频

难度 中

能力轴 Multi-Agent 通信

面试官在考什么 跨 Agent 复制全部历史既昂贵又扩大污染和泄露面。

30 秒回答

默认传最小充分上下文：目标、约束、已验证事实、必要 artifact 引用和未决问题。完整历史只在确有需要时附加检索入口。这样减少 token、隐私暴露和旧指令污染。

深挖解析

- handoff 是 context engineering 的一部分。
- 保留 provenance，避免摘要变成无来源事实。
- 不同 Agent 只获得完成其职责所需的信息。

连续追问

Q1 如何确定 ‘最小充分’ ？

Q2 摘要错误怎么发现？

易错回答 把全量 conversation dump 给每个 Agent。

Know-Why 跨 Agent 复制全部历史既昂贵又扩大污染和泄露面。

Know-How 定义 handoff schema：goal/constraints/facts/artifacts/open_questions；由 receiver 缺什么再按需取。

一句话收束 多 Agent 的核心不是角色数量，而是协议、状态、隔离和故障域。

37. 两个 Agent 对同一问题给出冲突结论怎么办？

频率 高频

难度 难

能力轴 Multi-Agent 通信

面试官在考什么 多 Agent 的数量不等于独立信息源。核心是证据质量和决策权边界。

30 秒回答

先比较证据和 authority，而不是多数投票。可以让 adjudicator 基于原始证据和 rubric 裁决；对不可确定问题显式保留分歧并升级人工。

深挖解析

- 置信度不能脱离证据校准。
- 不同 Agent 可能共享同一模型偏差。
- 投票适合独立、同质风险较低的判断，不适合事实权限冲突。

连续追问

Q1 Judge 也冲突怎么办？

Q2 如何降低 correlated error？

易错回答 三票两票自动决定真相。

Know-Why 多 Agent 的数量不等于独立信息源。核心是证据质量和决策权边界。

Know-How 让每个 Agent 返回 claim-evidence；裁决器按 source quality、freshness、policy 排序。

一句话收束 多 Agent 的核心不是角色数量，而是协议、状态、隔离和故障域。

38. 多个 Agent 相互调用进入无限 ‘甩锅循环’，怎么发现和阻断？

频率 必考

难度 难

能力轴 Multi-Agent 通信

面试官在考什么 多 Agent 的循环可能跨进程，单 Agent 局部看不见，因此必须有全局控制面。

30 秒回答

在运行时维护 agent call graph、depth/TTL、budget 和 repeated-edge signature；发现 A->B->A 等循环或无状态进展时终止 delegation，回到 orchestrator replan。

深挖解析

- 子 Agent 不应无限拥有 delegation 权。
- 调用深度和总预算分开控制。
- 循环检测要考虑不同参数的语义重复。

连续追问

Q1 合法递归任务怎么处理？

Q2 如何避免过早阻断？

易错回答 仅给每个 Agent 一个 max_steps，忽略跨 Agent 循环。

Know-Why 多 Agent 的循环可能跨进程，单 Agent 局部看不见，因此必须有全局控制面。

Know-How 每次 delegation 写 call edge；orchestrator 检查 depth、cycle、budget、progress，再签发 delegation token。

一句话收束 多 Agent 的核心不是角色数量，而是协议、状态、隔离和故障域。

39. 8 个 Worker 中一个失败，要不要整个任务失败？

频率 中频

难度 中

能力轴 Multi-Agent 通信

面试官在考什么 可靠性来自任务级故障域设计，而不是 worker 永不失败。

30 秒回答

看子任务的 criticality 和依赖。独立非关键任务可降级完成；关键路径任务可重试、重分配或用冗余 worker；需要 quorum 的任务按成功阈值决定。

深挖解析

- 任务图应标注 critical/optional。
- 部分失败必须显式体现在最终结果质量。
- 重分配时继承已有 artifact，避免从零重复。

连续追问

Q1 什么时候采用 speculative redundancy？

Q2 如何防止两个 Worker 同时写同一资源？

易错回答 任何 worker 失败都 fail-all，或无条件忽略失败。

Know-Why 可靠性来自任务级故障域设计，而不是 worker 永不失败。

Know-How 为每个 task 定义 retry_policy、fallback_worker、required_for_success；orchestrator 按 DAG 传播影响。

一句话收束 多 Agent 的核心不是角色数量，而是协议、状态、隔离和故障域。

40. 多 Agent 如何实现 Context、Storage、Permission 完全隔离，又允许必要交换？

频率 必考

难度 难

能力轴 Multi-Agent 通信

面试官在考什么 多 Agent 增加数据流路径，最容易出现跨上下文污染和越权。

30 秒回答

默认每个 Agent 拥有独立 context namespace、存储前缀和 capability token；跨 Agent 共享通过受控 handoff/artifact service，而不是直接访问对方内存。权限按 tenant、role、task 最小化。

深挖解析

- 隔离边界需要在存储和工具层强制执行。
- 共享对象应有 ACL 和 provenance。
- 不要把权限信息只放 prompt。

连续追问

Q1 共享向量库怎么做租户隔离？

Q2 同一 Worker 服务多个租户如何防串数据？

易错回答 依靠 Agent 名称或 system prompt 约束数据边界。

Know-Why 多 Agent 增加数据流路径，最容易出现跨上下文污染和越权。

Know-How 使用 tenant_id/task_id 贯穿 storage/query/tool token；artifact 分享通过显式 grant。

一句话收束 多 Agent 的核心不是角色数量，而是协议、状态、隔离和故障域。

CHAPTER 05

Context Engineering 与 Memory

上下文不是越多越好；真正要优化的是单位 token 的决策价值。

关键词 Context / Compaction / Memory / Provenance

本章训练目标

- 能在白板上画出关键控制边界，而不是只报框架名。
- 能从 failure mode 反推防护机制、状态和指标。
- 面对连续追问时，始终回到“为什么 + 怎么落地”。

41. 长任务 Context Window 快爆了怎么办？

频率 必考

难度 难

能力轴 Context Engineering

面试官在考什么 更长窗口缓解容量，不解决噪声和污染；有效 context engineering 优化的是决策相关性。

30 秒回答

把 context 当稀缺工作内存：保留目标、约束、当前状态和最近必要观察；历史工具结果外置，旧轨迹做 compaction，长期事实通过 retrieval 按需加载。必要时在 milestone 做 context reset。

深挖解析

- 区分 must-preserve state 与可压缩叙事。
- 工具大结果不要常驻 context。
- 压缩后仍保留原始 artifact 引用。

连续追问

Q1 Sliding window 会丢什么？

Q2 什么时候 reset 比 compaction 更好？

易错回答 只换一个更大上下文模型。

Know-Why 更长窗口缓解容量，不解决噪声和污染；有效 context engineering 优化的是决策相关性。

Know-How 实现 context builder：按 priority/token budget 拼装 system、goal、state、recent、retrieved、reserve。

一句话收束 Context 是工作内存，Memory 是长期状态；两者都需要显式治理。

42. Context Compaction 和 Context Reset 有什么区别？

频率 必考

难度 难

能力轴 Context Engineering

面试官在考什么 上下文连续性和状态连续性不是一回事。把状态外置后，模型上下文可以安全重建。

30 秒回答

Compaction 在同一执行连续性中压缩旧上下文；Reset 则开启新的上下文/session，只用结构化 handoff 接续。Compaction 保留更多连续性，Reset 对清除污染、长阶段切换更彻底。

深挖解析

- 两者都必须依赖外部 durable state。
- Reset 不是丢状态，而是丢不必要的语言轨迹。
- 阶段边界清晰时更适合 reset。

连续追问

Q1 Reset 会损失隐含信息怎么办？

Q2 多长时间做一次 compaction？

易错回答 把 reset 理解为清空一切重新开始。

Know-Why 上下文连续性和状态连续性不是一回事。把状态外置后，模型上下文可以安全重建。

Know-How 在 milestone checkpoint 生成 handoff package；新 session 从 package + artifact handles 启动。

一句话收束 Context 是工作内存，Memory 是长期状态；两者都需要显式治理。

43. 摘要会丢关键细节，如何保证压缩后 Agent 不失忆？

频率 必考

难度 难

能力轴 Context Engineering

面试官在考什么 压缩不可避免地有信息损失，因此必须把不可损失的信息移出自然语言压缩通道。

30 秒回答

不要让 summary 成为唯一事实源。关键状态用结构化字段/数据库无损保存；summary 只压缩叙事。对重要事实附 source reference、version、confidence，并支持按需回看原始记录。

深挖解析

- lossless state 与 lossy narrative 分离。
- 摘要可做 completeness check。
- 关键数值、ID、权限不能只留自然语言。

连续追问

Q1 如何验证摘要质量？

Q2 摘要模型也可能幻觉怎么办？

易错回答 把整段历史总结成一段话后删除原始数据。

Know-Why 压缩不可避免地有信息损失，因此必须把不可损失的信息移出自然语言压缩通道。

Know-How 定义 state schema；summary 只能引用 state，不负责创造 state；原始 artifact 保留可追溯句柄。

一句话收束 Context 是工作内存，Memory 是长期状态；两者都需要显式治理。

44. 什么是 Context Pollution? 如何判断 context 已经‘脏了’?

频率 高频

难度 中

能力轴 Context Engineering

面试官在考什么 Agent 决策基于可见上下文；错误信息即使只占少量 token 也可能成为高权重锚点。

30 秒回答

Context Pollution 指过时、无关、冲突、重复或恶意信息影响当前决策。信号包括模型反复引用旧状态、工具结果冲突、指令层级混淆、token 占用大但信息增益低。

深挖解析

- 污染来源包括旧计划、失败尝试、检索噪声和 prompt injection。
- context 越长不等于信息越多。
- 需要来源、freshness 和优先级。

连续追问

Q1 如何做自动 context hygiene?

Q2 旧失败轨迹是否应该删除?

易错回答 只看 token 是否超限，不看内容质量。

Know-Why Agent 决策基于可见上下文；错误信息即使只占少量 token 也可能成为高权重锚点。

Know-How context builder 对每块标记 source/type/freshness/priority；定期 prune stale/redundant content。

一句话收束 Context 是工作内存，Memory 是长期状态；两者都需要显式治理。

45. 100K token context budget 应该怎么分配？

频率 中频

难度 中

能力轴 Context Engineering

面试官在考什么 上下文选择是资源调度问题；目标是最大化每 token 的决策价值。

30 秒回答

先预留输出与工具调用空间，再给系统约束、目标、当前结构化状态最高优先级；其次是最近必要轨迹和相关检索；历史与低价值 tool output 最后。比例不是固定值，应按任务动态分配。

深挖解析

- token budget 是 runtime resource。
- 不同任务阶段优先级会变化。
- 可用 marginal utility 决定是否加入下一段上下文。

连续追问

Q1 如何知道某段 context 是否有价值？

Q2 预留 output budget 为什么重要？

易错回答 按固定 20/20/20/20/20 比例切。

Know-Why 上下文选择是资源调度问题；目标是最大化每 token 的决策价值。

Know-How 构建 context packer，按 priority score/token cost 做装箱，并记录被裁剪项供调试。

一句话收束 Context 是工作内存，Memory 是长期状态；两者都需要显式治理。

46. 多 Agent 是否应该共享同一个 Context?

频率 高频

难度 中

能力轴 Context Engineering

面试官在考什么 多 Agent 的价值之一就是独立上下文；全量共享会抵消这种优势并增加耦合。

30 秒回答

通常不应该全量共享。共享全局目标和已验证公共事实，角色专属历史和工具结果保持隔离；需要协作时通过 handoff/artifact 显式共享。

深挖解析

- 隔离能降低干扰和 token 成本。
- 公共状态必须有单一 source of truth。
- 共享越多，隐私和污染风险越大。

连续追问

Q1 哪些信息必须共享？

Q2 共享状态冲突如何处理？

易错回答 所有 Agent 读写同一个大 message buffer。

Know-Why 多 Agent 的价值之一就是独立上下文；全量共享会抵消这种优势并增加耦合。

Know-How 设计 public task state + private agent context；跨边界只能通过 typed message 或 artifact。

一句话收束 Context 是工作内存，Memory 是长期状态；两者都需要显式治理。

47. Agent 什么内容值得写入长期记忆？

频率 必考

难度 中

能力轴 Context Engineering

面试官在考什么 记忆错误会跨任务持续放大，比单轮幻觉更危险。

30 秒回答

不是所有对话都写。应保存跨会话仍有价值、稳定、用户允许、可验证的事实或偏好；临时任务细节、敏感信息、低置信推断通常不写或设置短 TTL。

深挖解析

- Memory 需要 write policy 与 read policy。
- 写入要有 provenance、timestamp、confidence。
- 支持用户纠正、删除和过期。

连续追问

Q1 如何判断 ‘稳定事实’ ？

Q2 模型推断出的偏好能不能写？

易错回答 所有消息向量化后永久存储。

Know-Why 记忆错误会跨任务持续放大，比单轮幻觉更危险。

Know-How 写入前经过 memory gate: utility、sensitivity、confidence、TTL、consent；低置信只做 episodic note。

一句话收束 Context 是工作内存，Memory 是长期状态；两者都需要显式治理。

48. 长期记忆出现两条互相冲突的信息怎么办？

频率 高频

难度 难

能力轴 Context Engineering

面试官在考什么 现实世界事实会变化；缺少 provenance 的 memory 无法判断新旧和可信度。

30 秒回答

不要静默覆盖。保留版本、时间、来源和置信度，定义 supersession 规则；涉及用户事实时可询问确认。读取时按 freshness、authority 和任务相关性决策。

深挖解析

- Memory 是 versioned knowledge，而不是 key-value 永久真相。
- 冲突本身也是信息。
- 对不可判定冲突返回 uncertainty。

连续追问

Q1 用户今天和昨天的偏好不同怎么办？

Q2 企业事实谁有更高 authority？

易错回答 最后写入覆盖前一条，不保留历史。

Know-Why 现实世界事实会变化；缺少 provenance 的 memory 无法判断新旧和可信度。

Know-How memory record 加 valid_from/valid_to/source/confidence/supersedes；读取时做 conflict resolution。

一句话收束 Context 是工作内存，Memory 是长期状态；两者都需要显式治理。

49. Semantic、Episodic、Procedural Memory 分别解决什么问题？

频率 中频

难度 中

能力轴 Context Engineering

面试官在考什么 不同记忆类型的‘相似度’和‘新鲜度’含义不同，混在一个向量库会难以治理。

30 秒回答

Semantic 保存可复用事实知识；Episodic 保存具体经历与过去任务；Procedural 保存如何做事的策略/技能。三者读写频率、结构和更新策略不同。

深挖解析

- Semantic 更接近知识库。
- Episodic 用于经验回忆和案例检索。
- Procedural 应谨慎更新，避免错误经验固化。

连续追问

Q1 聊天记录属于哪一种？

Q2 Skill 文件算 procedural memory 吗？

易错回答 只背定义，不说明工程存储与检索方式。

Know-Why 不同记忆类型的‘相似度’和‘新鲜度’含义不同，混在一个向量库会难以治理。

Know-How 分别定义 schema/index/retrieval policy；统一通过 memory router 按任务查询。

一句话收束 Context 是工作内存，Memory 是长期状态；两者都需要显式治理。

50. 怎么证明你的 Memory 系统真的有效？

频率 必考

难度 难

能力轴 Context Engineering

面试官在考什么 Memory 的目标是改善任务行为，不是把数据库查得更像。

30 秒回答

做有/无 Memory 的对照 eval，观察 task success、重复询问率、个性化准确率，同时测 retrieval precision、stale/contradiction rate 和隐私错误。只看命中率不够。

深挖解析

- Memory 可能提高体验也可能引入错误。
- 需要评估写入、检索、使用三个阶段。
- 对长期效果做时间切片回放。

连续追问

Q1 Memory 命中了但模型没用怎么办？

Q2 怎样构造 memory benchmark？

易错回答 只统计向量检索 Recall@K。

Know-Why Memory 的目标是改善任务行为，不是把数据库查得更像。

Know-How 建立 memory eval set：需记忆/不该记忆/冲突/过时/敏感五类；比较 end-to-end delta。

一句话收束 Context 是工作内存，Memory 是长期状态；两者都需要显式治理。

CHAPTER 06

Agentic RAG

RAG 在 Agent 中不只影响答案，还会影响下一步行动，因此检索错误会放大成执行错误。

关键词 Retrieval / Hybrid / Rerank / ACL / Eval

本章训练目标

- 能在白板上画出关键控制边界，而不是只报框架名。
- 能从 failure mode 反推防护机制、状态和指标。
- 面对连续追问时，始终回到“为什么 + 怎么落地”。

51. Agent 中 RAG 应该一直执行，还是作为 Tool 按需调用？

频率 高频

难度 中

能力轴 Agentic RAG

面试官在考什么 检索触发决定了知识可靠性和成本平衡。

30 秒回答

取决于任务知识依赖。强知识型问答可 always retrieve；开放 Agent 更适合把检索作为可调用能力，但要有 retrieval trigger，避免模型因自信而跳过必要检索。

深挖解析

- 检索本身有 latency/cost。
- 高风险、时效事实可强制 retrieval。
- 触发策略可以规则+模型混合。

连续追问

Q1 怎么防止模型不检索直接回答？

Q2 哪些问题可以不检索？

易错回答 默认每轮 top-k，或完全交给模型自由判断。

Know-Why 检索触发决定了知识可靠性和成本平衡。

Know-How 按 domain/risk/freshness 定义 mandatory retrieval；其他场景由 router 判断并记录 decision reason。

一句话收束 RAG 在 Agent 中是行动依据，因此证据质量必须进入执行策略。

52. 模型怎么判断自己现在需要检索？

频率 高频

难度 中

能力轴 Agentic RAG

面试官在考什么 RAG 的第一步不是检索算法，而是是否检索。漏检会导致无依据生成，过检索则浪费资源。

30 秒回答

可以用规则、分类器、模型自评或混合策略。可靠方案通常先用确定性规则覆盖高风险/实时问题，再让模型判断开放场景；必要时用 uncertainty 和 evidence requirement 触发。

深挖解析

- self-confidence 未必校准。
- 触发器要单独做 precision/recall eval。
- 过度检索与漏检都要计成本。

连续追问

Q1 Retrieval trigger 的标签怎么标？

Q2 触发器错了怎么兜底？

易错回答 直接问模型 ‘你需要搜索吗？’ 并完全信任。

Know-Why RAG 的第一步不是检索算法，而是是否检索。漏检会导致无依据生成，过检索则浪费资源。

Know-How 构造 must-retrieve / optional / no-retrieve 数据集，评估触发准确率，再联合端到端指标调阈值。

一句话收束 RAG 在 Agent 中是行动依据，因此证据质量必须进入执行策略。

53. Chunk 越大越好吗？

频率 高频

难度 中

能力轴 Agentic RAG

面试官在考什么 Chunk 是检索单元，也是生成上下文单元，影响精确率和完整性两个方向。

30 秒回答

不是。大 chunk 保留语义完整但噪声多、成本高；小 chunk 精准但可能丢上下文。应结合文档结构、问题粒度、embedding 模型和 rerank 方式，通过 retrieval+generation eval 选择。

深挖解析

- 固定字符切分往往破坏语义边界。
- 可用 parent-child / small-to-big retrieval。
- chunk 参数必须以任务评估为依据。

连续追问

Q1 代码文档怎么切？

Q2 表格和 PDF 怎么切？

易错回答 只凭经验固定 500 或 1000 tokens。

Know-Why Chunk 是检索单元，也是生成上下文单元，影响精确率和完整性两个方向。

Know-How 先按标题/段落/语义切分，网格搜索 chunk_size/overlap/top_k，联合评估 Recall 与 answer quality。

一句话收束 RAG 在 Agent 中是行动依据，因此证据质量必须进入执行策略。

54. 为什么生产 RAG 常采用 Hybrid Retrieval + Rerank?

频率 必考

难度 中

能力轴 Agentic RAG

面试官在考什么 生产知识包含语义、字面、结构等多种信号，单一检索范式容易漏召。

30 秒回答

向量检索擅长语义相似，BM25 擅长关键词、编号和专有名词；Hybrid 提高召回，Rerank 再用更强相关性模型压缩候选，提高最终 precision。

深挖解析

- 第一阶段追求 recall，第二阶段追求 precision。
- 融合可用 RRF 或加权分数。
- rerank 的输入规模受 latency budget 限制。

连续追问

Q1 Hybrid 一定优于纯向量吗？

Q2 Reranker 选 cross-encoder 还是 LLM？

易错回答 把所有改进都归因于 embedding 模型更强。

Know-Why 生产知识包含语义、字面、结构等多种信号，单一检索范式容易漏召。

Know-How 建立 query taxonomy；分别统计各路召回贡献与 rerank lift，再做按类型动态融合。

一句话收束 RAG 在 Agent 中是行动依据，因此证据质量必须进入执行策略。

55. Retriever 找错资料，Agent 根据错误资料执行错误动作，怎么解决？

频率 必考

难度 难

能力轴 Agentic RAG

面试官在考什么 错误知识进入 Agent 后会转化为真实动作，blast radius 高于普通问答。

30 秒回答

Agentic RAG 要把检索证据当作不可信输入。执行前检查 source authority、freshness、cross-source consistency；高风险动作要求多证据或业务 API 验证。检索证据不足时禁止直接产生副作用。

深挖解析

- 答案错误和动作错误的风险级别不同。
- RAG evidence 需要 provenance。
- 使用 action gate 将知识置信度映射到执行权限。

连续追问

Q1 怎样评估 evidence sufficiency?

Q2 多个来源互相冲突怎么办？

易错回答 只优化 Recall@K，认为检索到 top1 就可信。

Know-Why 错误知识进入 Agent 后会转化为真实动作，blast radius 高于普通问答。

Know-How tool precondition 中加入 evidence policy：source tier、age、cross-check、confidence；不足则 ask/HITL。

一句话收束 RAG 在 Agent 中是行动依据，因此证据质量必须进入执行策略。

56. Agent 如何做 Multi-hop / Iterative Retrieval?

频率 高频

难度 难

能力轴 Agentic RAG

面试官在考什么 复杂问题的答案往往分散在多个实体或文档中，单次相似度检索难覆盖完整链路。

30 秒回答

先检索一轮，识别已知/未知和证据缺口，再生成下一跳 query；每轮把新证据并入 evidence graph，直到问题被覆盖或达到检索预算。

深挖解析

- 下一跳 query 应由 evidence gap 驱动。
- 避免反复检索同一内容。
- 对每条 claim 保留支持证据。

连续追问

Q1 怎么决定停止检索？

Q2 如何避免 query drift？

易错回答 一次 top-k 后让模型自己拼答案。

Know-Why 复杂问题的答案往往分散在多个实体或文档中，单次相似度检索难覆盖完整链路。

Know-How 维护 claim/evidence graph；每轮只为未覆盖 claim 生成 query，并做 dedup + max_hops。

一句话收束 RAG 在 Agent 中是行动依据，因此证据质量必须进入执行策略。

57. 有 ACL 的企业知识库，是 retrieval 前过滤还是 retrieval 后过滤？

频率 必考

难度 难

能力轴 Agentic RAG

面试官在考什么 只要无权限内容进入模型上下文，就已经越过了安全边界。

30 秒回答

安全边界必须在检索层或数据层强制过滤，不能先取出无权限文档再依靠模型不展示。可在向量/关键词检索中加入 tenant/user ACL filter，后续 rerank 只处理授权候选。

深挖解析

- 权限过滤应尽量早。
- embedding 索引也要避免跨租户泄漏。
- 日志与缓存同样要按权限隔离。

连续追问

Q1 过滤影响召回怎么办？

Q2 共享文档权限变化如何更新索引？

易错回答 检索全库后，在 prompt 里要求模型不要泄露。

Know-Why 只要无权限内容进入模型上下文，就已经越过了安全边界。

Know-How 索引记录携带 tenant_id/acl_version；query 必须注入可信 identity filter，并做审计。

一句话收束 RAG 在 Agent 中是行动依据，因此证据质量必须进入执行策略。

58. 知识库一分钟更新一次，但向量索引十分钟更新一次，Agent 如何避免读旧数据？

频率 高频

难度 难

能力轴 Agentic RAG

面试官在考什么 RAG 引入了新的数据延迟层，实时性要求必须显式设计。

30 秒回答

把 freshness 作为检索元数据和业务约束。对强实时数据优先调用 source API；RAG 返回 index_version/updated_at，若超出 freshness SLA 则降级到实时源或明确提示。

深挖解析

- 知识源有 source of truth 与 derived index 之分。
- 索引延迟必须可观测。
- 缓存也要参与 freshness 判断。

连续追问

Q1 如何做增量索引？

Q2 旧数据被删除但索引没同步怎么办？

易错回答 默认向量库就是最新真相。

Know-Why RAG 引入了新的数据延迟层，实时性要求必须显式设计。

Know-How 为每个 domain 定义 freshness SLA；retriever 返回 version；超阈值自动查询原系统或阻断高风险回答。

一句话收束 RAG 在 Agent 中是行动依据，因此证据质量必须进入执行策略。

59. RAG 到底怎么评估?

频率 必考

难度 难

能力轴 Agentic RAG

面试官在考什么 分层 eval 才能做 failure attribution 和定向优化。

30 秒回答

至少拆成三层：检索层测 Recall@K、MRR/nDCG、ground-truth coverage；生成层测 faithfulness、relevance、citation correctness；端到端看真实 task success、latency、cost 和 failure type。

深挖解析

- 检索好不代表生成一定好。
- 离线指标需要和线上业务指标校准。
- 按 query taxonomy 分层统计。

连续追问

Q1 没有标准答案怎么办？

Q2 LLM judge 可以评 RAG 吗？

易错回答 只看最终答案准确率，无法知道错在哪层。

Know-Why 分层 eval 才能做 failure attribution 和定向优化。

Know-How 建立 golden queries + judged docs；每次索引、embedding、rerank、prompt 变更跑 regression suite。

一句话收束 RAG 在 Agent 中是行动依据，因此证据质量必须进入执行策略。

60. 向量数据库挂了，Agent 是直接失败还是降级？

频率 高频

难度 中

能力轴 Agentic RAG

面试官在考什么 故障模式需要提前设计，否则事故现场才临时决策会放大风险。

30 秒回答

看业务关键性和替代源。可降级到 BM25/搜索、缓存、源 API 或让用户等待/缩小任务；但不能静默用旧或无依据结果冒充正常。降级路径也要有质量标识和 SLO。

深挖解析

- Fallback 必须在正常时验证。
- 不同业务定义不同 degraded mode。
- 高风险场景可选择 fail-closed。

连续追问

Q1 什么时候应该 fail-open？

Q2 缓存过期还能用吗？

易错回答 异常时随便换一个检索方式且不做提示。

Know-Why 故障模式需要提前设计，否则事故现场才临时决策会放大风险。

Know-How 定义 retriever circuit breaker + fallback chain；每条路径附 mode/source/freshness，trace 中可区分。

一句话收束 RAG 在 Agent 中是行动依据，因此证据质量必须进入执行策略。

CHAPTER 07

Durable Execution 与 Fault Tolerance

长任务要像 workflow 引擎一样可暂停、可恢复、可幂等、可补偿。

关键词 Checkpoint / Retry / Saga / Versioning

本章训练目标

- 能在白板上画出关键控制边界，而不是只报框架名。
- 能从 failure mode 反推防护机制、状态和指标。
- 面对连续追问时，始终回到“为什么 + 怎么落地”。

61. 一个 Agent 运行 40 分钟，第 39 分钟进程 crash，怎么办？

频率 必考

难度 难

能力轴 Durable Execution

面试官在考什么 长任务遇到进程、网络、部署故障是常态，可靠性来自恢复能力而非不出错。

30 秒回答

不能从头重跑。把执行拆成可 checkpoint 的幂等步骤，持久化 state、已完成任务和外部 operation id；新 worker 从最近 checkpoint 恢复，并只执行未完成或未确认的步骤。

深挖解析

- Durable execution 要求状态不依赖进程内存。
- 恢复前要 reconciliation 外部副作用。
- checkpoint 粒度影响恢复成本和写放大。

连续追问

Q1 多长时间 checkpoint 一次？

Q2 模型调用中间 crash 能恢复吗？

易错回答 让用户重新开始或依赖进程一直不死。

Know-Why 长任务遇到进程、网络、部署故障是常态，可靠性来自恢复能力而非不出错。

Know-How 每个 side-effect boundary 前后 checkpoint；worker stateless 化；run_id 可被任意实例 resume。

一句话收束 可靠性来自“允许失败但能恢复”，而不是假设执行永不崩溃。

62. Checkpoint 应该保存什么？

频率 必考

难度 中

能力轴 Durable Execution

面试官在考什么 恢复需要知道‘世界已经发生了什么’，而不只是模型看过什么。

30 秒回答

至少保存可恢复所需的 control state、business state 引用、已完成步骤、pending action、工具结果/operation id、计划版本、预算和 schema version。不是简单 dump 全部 prompt。

深挖解析

- 关键是足够恢复且可版本化。
- 大 artifact 存引用，不必内嵌。
- checkpoint 要原子写入或可判定状态。

连续追问

Q1 保存模型 hidden reasoning 吗？

Q2 Checkpoint 和 event log 有什么区别？

易错回答 把聊天历史序列化就叫 checkpoint。

Know-Why 恢复需要知道‘世界已经发生了什么’，而不只是模型看过什么。

Know-How 定义 checkpoint schema；事件流记录每次 transition，checkpoint 保存最近可恢复快照，两者结合 replay。

一句话收束 可靠性来自“允许失败但能恢复”，而不是假设执行永不崩溃。

63. Tool 已执行成功，但 Agent 在写 checkpoint 前 crash，会发生什么？

频率 必考

难度 难

能力轴 Durable Execution

面试官在考什么 这是 Agent 工程最典型的分布式一致性问题之一。

30 秒回答

恢复后可能重复执行。这是 side effect 与本地状态提交之间的 crash-consistency 问题。解决依赖 operation/idempotency key、outbox/transactional state、状态查询和 reconciliation，而不是单纯缩短 checkpoint 间隔。

深挖解析

- exactly-once 通常需要业务幂等。
- 外部系统返回的 operation id 必须先持久化。
- 无法幂等时考虑 compensation。

连续追问

Q1 本地 DB 和第三方 API 无法做分布式事务怎么办？

Q2 Outbox 模式怎么用？

易错回答 认为 checkpoint 每秒一次就不会重复。

Know-Why 这是 Agent 工程最典型的分布式一致性问题之一。

Know-How 执行前持久化 INTENT+operation_id；外部执行后按同 ID 查询/确认；最终提交 DONE。

一句话收束 可靠性来自“允许失败但能恢复”，而不是假设执行永不崩溃。

64. 哪些错误应该 retry，哪些绝对不应该 retry？

频率 必考

难度 中

能力轴 Durable Execution

面试官在考什么 错误类型不同，重试可能修复问题，也可能扩大副作用或拖垮下游。

30 秒回答

按错误语义分类：网络瞬断、429、部分 5xx 可受控重试；参数校验、权限、业务拒绝、不可逆失败不应盲重试。还要考虑操作幂等性和剩余 deadline。

深挖解析

- 错误码需要携带 retryable。
- Retry budget 防止重试风暴。
- 使用 exponential backoff + jitter。

连续追问

Q1 LLM 输出错误算 retryable 吗？

Q2 429 的 retry-after 怎么处理？

易错回答 所有 exception 统一 retry 3 次。

Know-Why 错误类型不同，重试可能修复问题，也可能扩大副作用或拖垮下游。

Know-How 建立 error taxonomy -> policy table: retry/backoff/fallback/HITL/fail-fast。

一句话收束 可靠性来自“允许失败但能恢复”，而不是假设执行永不崩溃。

65. Agent 为什么需要 Circuit Breaker?

频率 高频

难度 中

能力轴 Durable Execution

面试官在考什么 自治系统会自动重复尝试，因此比人工调用更容易形成故障放大器。

30 秒回答

当下游持续失败时，Agent 的自动重试会放大流量。Circuit Breaker 在错误率/超时达到阈值后暂时拒绝新调用，进入 fallback 或快速失败，待探测恢复后再关闭。

深挖解析

- 状态通常 CLOSED/OPEN/HALF_OPEN。
- 按 tool/downstream/tenant 粒度设计。
- 与 retry budget 配合。

连续追问

Q1 阈值怎么设？

Q2 多个实例如何共享 breaker 状态？

易错回答 只靠每个请求重试和 timeout。

Know-Why 自治系统会自动重复尝试，因此比人工调用更容易形成故障放大器。

Know-How 在 Tool Gateway 记录 rolling error rate；OPEN 时返回结构化 degraded result 并触发 fallback。

一句话收束 可靠性来自“允许失败但能恢复”，而不是假设执行永不崩溃。

66. 整个 Agent SLA 30 秒，内部 5 个工具 timeout 怎么分？

频率 高频

难度 中

能力轴 Durable Execution

面试官在考什么 局部 timeout 不考虑全局 deadline 会导致尾延迟叠加。

30 秒回答

使用 end-to-end deadline 而不是每个工具独立 30 秒。根据关键路径和历史 P95 分配预算，调用时传播剩余 deadline；预算不足时跳过非关键步骤或降级。

深挖解析

- timeout budget 应包含模型和排队时间。
- 并行调用使用共同 deadline。
- 不要让单个子调用耗尽全部 SLA。

连续追问

Q1 动态预算怎么做？

Q2 慢工具但高价值怎么办？

易错回答 每个工具都设 30 秒超时。

Know-Why 局部 timeout 不考虑全局 deadline 会导致尾延迟叠加。

Know-How run_context 保存 deadline_at；每次调用计算 remaining_ms 并设置子 timeout + reserve。

一句话收束 可靠性来自“允许失败但能恢复”，而不是假设执行永不崩溃。

67. 流量突然增长 20 倍，Agent 队列怎么保护系统？

频率 高频

难度 难

能力轴 Durable Execution

面试官在考什么 高并发下最重要的是限制负载进入，而不是让每个请求都慢慢失败。

30 秒回答

先 admission control 和 per-tenant quota，队列设置长度/等待时间上限；下游按并发和 rate limit 做 backpressure，必要时降级模型、减少 tool depth 或拒绝低优先级任务。

深挖解析

- Agent 任务耗时长，队列容量比普通 API 更敏感。
- 优先级调度避免大任务饿死小任务。
- 监控 queue age 而不只看 queue length。

连续追问

Q1 如何做公平调度？

Q2 排队超时后任务状态怎么处理？

易错回答 无限排队，认为系统总会处理完。

Know-Why 高并发下最重要的是限制负载进入，而不是让每个请求都慢慢失败。

Know-How 设置 token-bucket quota、max queue age、worker concurrency；过载触发 degraded policy。

一句话收束 可靠性来自“允许失败但能恢复”，而不是假设执行永不崩溃。

68. 同一用户同时启动两个修改同一资源的 Agent，怎么办？

频率 高频

难度 难

能力轴 Durable Execution

面试官在考什么 Agent 的执行时间长，更容易发生并发冲突和陈旧读取。

30 秒回答

需要并发控制。可用资源级锁、乐观版本号/ETag、事务或 compare-and-swap；冲突时让后提交者 re-read/replan，而不是最后写入覆盖。

深挖解析

- Agent 长事务不宜一直持锁。
- 外部系统版本号是更可靠的冲突检测依据。
- 冲突本身要反馈给 planner。

连续追问

Q1 锁超时怎么办？

Q2 多个资源跨系统更新怎么处理？

易错回答 相信同一用户不会同时发两个任务。

Know-Why Agent 的执行时间长，更容易发生并发冲突和陈旧读取。

Know-How 读取资源时保存 version；写入时带 expected_version；冲突返回 typed error 触发 replan。

一句话收束 可靠性来自“允许失败但能恢复”，而不是假设执行永不崩溃。

69. Agent 已执行 A、B、C、D 失败，需要 rollback，怎么设计？

频率 必考

难度 难

能力轴 Durable Execution

面试官在考什么 Agent 常跨多个系统，分布式事务不可得，业务补偿是更现实的可靠性机制。

30 秒回答

外部副作用通常无法真正 ACID 回滚，应采用 Saga/Compensation：为每个可逆步骤定义补偿动作，失败时按逆序执行；对不可逆步骤设计审批点或前置验证，避免进入后才想回滚。

深挖解析

- 补偿不是数据库 rollback，可能失败。
- 补偿动作也必须幂等、可追踪。
- 不可逆动作要尽量放在最后。

连续追问

Q1 补偿失败怎么办？

Q2 什么时候选择 roll-forward 而非 rollback？

易错回答 认为可以把所有外部 API 包进一个事务。

Know-Why Agent 常跨多个系统，分布式事务不可得，业务补偿是更现实的可靠性机制。

Know-How TaskGraph 标记 compensation handler；失败触发 saga state machine，并对失败补偿升级人工。

一句话收束 可靠性来自“允许失败但能恢复”，而不是假设执行永不崩溃。

70. v1 代码产生的 checkpoint, 在 v2 发布后如何恢复?

频率 高频

难度 难

能力轴 Durable Execution

面试官在考什么 Durable execution 把运行时状态变成长期数据，因此必须像数据库 schema 一样管理版本。

30 秒回答

Checkpoint 必须带 schema/runtime version。新版本读取旧状态时经过 migration/compatibility layer；无法兼容则继续由旧 worker 完成或显式终止，而不是直接反序列化碰运气。

深挖解析

- 长任务跨部署版本很常见。
- 状态迁移要可回滚。
- 工具和 prompt 版本也影响 replay 可重复性。

连续追问

Q1 蓝绿发布怎么处理长 run?

Q2 Replay 需要固定模型版本吗?

易错回答 默认发布后所有旧 run 自动适配新代码。

Know-Why Durable execution 把运行时状态变成长期数据，因此必须像数据库 schema 一样管理版本。

Know-How checkpoint 加 schema_version/runtime_version；维护 migration registry；部署时做 can-resume 测试。

一句话收束 可靠性来自“允许失败但能恢复”，而不是假设执行永不崩溃。

CHAPTER 08

Evaluation、Tracing 与 Observability

不能观测的 Agent 无法工程化；不能归因的失败无法系统优化。

关键词 Trace / Eval / Regression / SLO

本章训练目标

- 能在白板上画出关键控制边界，而不是只报框架名。
- 能从 failure mode 反推防护机制、状态和指标。
- 面对连续追问时，始终回到“为什么 + 怎么落地”。

71. Agent 每次 Run 到底应该 Trace 什么？

频率 必考

难度 中

能力轴 Evaluation、Tracing

面试官在考什么 Agent 错误可能发生在任意中间步骤，没有 trajectory 就无法归因。

30 秒回答

至少有 run/task 根 span，并记录 model generation、tool call、retrieval、handoff、guardrail、checkpoint、retry 和 verifier。每个 span 带输入摘要、输出摘要、latency、token/cost、error_type 和 correlation id。

深挖解析

- Trace 是 trajectory 的可重放证据。
- 敏感数据要脱敏/采样。
- 模型和工具版本必须记录。

连续追问

Q1 Trace 全量存会不会太贵？

Q2 如何关联多 Agent 跨服务 span？

易错回答 只打印 prompt 和 final answer。

Know-Why Agent 错误可能发生在任意中间步骤，没有 trajectory 就无法归因。

Know-How 采用 OpenTelemetry 类似 span 模型；run_id/task_id/correlation_id 全链路透传，按风险与错误采样。

一句话收束 先找最早的坏 transition，再决定改模型、RAG、Tool 还是 Runtime。

72. 线上 Agent 成功率从 85% 降到 70%，怎么定位？

频率 必考

难度 难

能力轴 Evaluation、Tracing

面试官在考什么 聚合成功率无法告诉你根因；分层归因是生产调试的第一步。

30 秒回答

先做 failure segmentation，而不是立刻改 prompt：按模型版本、任务类型、tool、retrieval、tenant、时间、error taxonomy 切片，找变化最大的层；再抽取失败 trace 与历史基线比较。

深挖解析

- 检查输入分布和下游依赖是否变化。
- 区分模型质量下降与 infra/tool 失败。
- 看 success denominator 是否变化。

连续追问

Q1 如果所有层都轻微恶化怎么办？

Q2 如何快速止血？

易错回答 随机看几个 bad case 后改 system prompt。

Know-Why 聚合成功率无法告诉你根因；分层归因是生产调试的第一步。

Know-How 建立 dashboard：task success -> failure type -> component -> version；支持一键 drill-down 到 trace。

一句话收束 先找最早的坏 transition，再决定改模型、RAG、Tool 还是 Runtime。

73. Agent Eval Dataset 怎么构建?

频率 必考

难度 中

能力轴 Evaluation、Tracing

面试官在考什么 Agent 的主要问题常在边界与组合条件，单纯静态问答集覆盖不了。

30 秒回答

覆盖真实高频任务、长尾 edge case、工具失败、歧义、多轮、对抗输入和长 horizon；每条样本包含目标、初始环境、可接受结果与评分 rubric。用生产失败持续回流形成 evolving eval set。

深挖解析

- 训练样本与评测样本隔离。
- 按能力轴分层，不只一个总分。
- 保留 deterministic fixtures 便于 replay。

连续追问

Q1 多少条 eval 才够？

Q2 如何避免评测集过拟合？

易错回答 只用十几个 happy path prompt。

Know-Why Agent 的主要问题常在边界与组合条件，单纯静态问答集覆盖不了。

Know-How 从生产 trace 聚类失败模式，每个 cluster 建 representative case；每次修复把 case 加入 regression。

一句话收束 先找最早的坏 transition，再决定改模型、RAG、Tool 还是 Runtime。

74. 为什么 Agent 不能只评最终答案？

频率 必考

难度 难

能力轴 Evaluation、Tracing

面试官在考什么 Agent 的价值在行动；错误轨迹可能已经造成不可逆副作用，即使最终文本看起来正确。

30 秒回答

同样的最终答案可能来自高风险或低效轨迹；Agent 还会调用工具、修改状态，因此要同时评 trajectory：工具是否正确、是否越权、步骤是否冗余、证据是否充分、是否触发不必要副作用。

深挖解析

- final success 与 process quality 分离。
- 一些错误最终被偶然修复，但仍暴露系统风险。
- 轨迹指标可用于定位优化。

连续追问

Q1 如何评价‘合理轨迹’而不限制创新路径？

Q2 哪些步骤必须 deterministic 检查？

易错回答 只要最后答对就算成功。

Know-Why Agent 的价值在行动；错误轨迹可能已经造成不可逆副作用，即使最终文本看起来正确。

Know-How 定义 hard invariants + soft trajectory rubric；对高风险动作硬判，对路径效率用统计评分。

一句话收束 先找最早的坏 transition，再决定改模型、RAG、Tool 还是 Runtime。

75. LLM-as-a-Judge 有什么问题？

频率 高频

难度 中

能力轴 Evaluation、Tracing

面试官在考什么 评估器如果不稳定，所有优化决策都会被噪声误导。

30 秒回答

Judge 可能有自偏好、位置偏差、风格偏差、随机性和与 actor 的相关错误。应使用明确 rubric、隐藏模型身份、随机顺序、多个 judge/人工校准，并对关键指标尽量引入可执行验证。

深挖解析

- Judge 也要做 reliability eval。
- 不要把单次 judge score 当客观真值。
- 可计算 inter-rater agreement。

连续追问

Q1 同模型做 actor 和 judge 可不可以？

Q2 什么时候必须人工评？

易错回答 把 LLM Judge 当作万能 ground truth。

Know-Why 评估器如果不稳定，所有优化决策都会被噪声误导。

Know-How 先用人工标注小集校准 judge；记录 agreement、false positive/negative，关键任务加 deterministic checks。

一句话收束 先找最早的坏 transition，再决定改模型、RAG、Tool 还是 Runtime。

76. 修改一句 System Prompt，怎么确定没有让其他任务退化？

频率 必考

难度 中

能力轴 Evaluation、Tracing

面试官在考什么 LLM 行为高度耦合，局部提示修改可能改变其他能力，因此必须回归测试。

30 秒回答

跑版本化 regression suite，至少比较 task success、tool selection、safety、latency、cost 和 failure distribution；对显著差异做 paired analysis，而不是只看平均分。

深挖解析

- Prompt 也是代码，要版本化。
- 不同任务可能 trade-off。
- 线上变更先 canary。

连续追问

Q1 Eval 有波动怎么判断显著？

Q2 如何做 prompt rollback？

易错回答 只测试刚修复的那个 case。

Know-Why LLM 行为高度耦合，局部提示修改可能改变其他能力，因此必须回归测试。

Know-How prompt_version 写入 trace；CI 中跑固定 eval；上线 5% canary，指标异常自动回滚。

一句话收束 先找最早的坏 transition，再决定改模型、RAG、Tool 还是 Runtime。

77. Agent 在线 A/B Test 应该观察什么？

频率 高频

难度 中

能力轴 Evaluation、Tracing

面试官在考什么 Agent 的收益和代价同时发生，单指标可能掩盖严重退化。

30 秒回答

主指标看任务成功/业务转化；同时看 P95 latency、token/cost、tool call 数、error、人工升级率、安全事件和用户重试。Agent 优化是多目标，不能只追一个质量分。

深挖解析

- 实验单元要避免同用户跨组污染。
- 长任务需考虑 delayed outcome。
- 按任务类型分层分析。

连续追问

Q1 如何定义 guardrail metric？

Q2 新版本质量更高但成本翻倍怎么办？

易错回答 只比较用户点赞率。

Know-Why Agent 的收益和代价同时发生，单指标可能掩盖严重退化。

Know-How 建立 scorecard：quality/reliability/latency/cost/safety；预先定义 non-inferiority guardrails。

一句话收束 先找最早的坏 transition，再决定改模型、RAG、Tool 还是 Runtime。

78. 怎么在线发现一个 Agent 已经开始 runaway?

频率 必考

难度 难

能力轴 Evaluation、Tracing

面试官在考什么 高步数只是结果，真正危险是‘消耗增长但状态无进展’。

30 秒回答

监控 step rate、token velocity、重复 action signature、错误重试率、相同状态哈希和外部调用频率；达到动态阈值时暂停 run，触发 replan/circuit breaker 或人工。

深挖解析

- runaway 是轨迹异常，不只是总步数大。
- 不同任务类型阈值不同。
- 需要实时 stream metrics。

连续追问

Q1 合法长任务会误报怎么办？

Q2 发现 runaway 后怎么保留现场？

易错回答 只设置 max_steps=100。

Know-Why 高步数只是结果，真正危险是‘消耗增长但状态无进展’。

Know-How 每步计算 progress + action novelty；无进展窗口超过阈值冻结执行并保存 checkpoint/trace。

一句话收束 先找最早的坏 transition，再决定改模型、RAG、Tool 还是 Runtime。

79. 用户说 ‘Agent 答错了’，怎么判断是 Model、RAG、Tool 还是 Planner 的问题？

频率 必考

难度 难

能力轴 Evaluation、Tracing

面试官在考什么 只有找到 first bad transition，修复才可能有针对性。

30 秒回答

沿 trajectory 做 failure attribution：先看目标解析和计划是否正确，再看检索证据是否支持、工具调用参数/结果是否正确、模型是否正确使用观察，最后验证状态与完成判定。用反事实 replay 定位最早发生偏离的步骤。

深挖解析

- 最早错误点通常比最终错误更重要。
- 组件级指标与 end-to-end trace 联动。
- 可替换单组件做 counterfactual test。

连续追问

Q1 如何自动归因？

Q2 多个组件共同导致怎么办？

易错回答 把错误直接归因于 LLM hallucination。

Know-Why 只有找到 first bad transition，修复才可能有针对性。

Know-How 实现 debug replay：固定其余输入，只替换 retrieval/tool/model step，观察 final outcome 是否恢复。

一句话收束 先找最早的坏 transition，再决定改模型、RAG、Tool 还是 Runtime。

80. Agent 的 SLO 应该怎么定义？

频率 高频

难度 难

能力轴 Evaluation、Tracing

面试官在考什么 Agent 是复合系统，任一组件成功都不代表用户任务成功。

30 秒回答

至少包括可用性、端到端 task success、P95/P99 latency、cost/task、unsafe action rate 和恢复率；对长任务还可定义 resume success 与 deadline miss。SLO 要按任务等级拆分。

深挖解析

- 模型 API availability 不等于 Agent availability。
- 质量 SLO 需要可评估的任务定义。
- 预算与安全也可成为 SLO/guardrail。

连续追问

Q1 创意 Agent 的 task success 怎么定义？

Q2 SLO 和 SLA 有什么区别？

易错回答 只定义 HTTP 99.9% 可用。

Know-Why Agent 是复合系统，任一组件成功都不代表用户任务成功。

Know-How 为关键 user journey 定义 SLI；按 severity 设置 error budget，并用 error budget 驱动发布节奏。

一句话收束 先找最早的坏 transition，再决定改模型、RAG、Tool 还是 Runtime。

CHAPTER 09

Security、Permission 与 HITL

Prompt 不是安全边界；高风险动作必须在可信执行层被限制。

关键词 Injection / Least Privilege / HITL / Sandbox

本章训练目标

- 能在白板上画出关键控制边界，而不是只报框架名。
- 能从 failure mode 反推防护机制、状态和指标。
- 面对连续追问时，始终回到“为什么 + 怎么落地”。

81. 网页里的 Prompt Injection 告诉 Agent ‘忽略之前指令’，怎么办？

频率 必考

难度 难

能力轴 Security、Permission

面试官在考什么 Prompt injection 的根因是模型同时处理指令和数据，而传统文本边界并非真正安全边界。

30 秒回答

把外部内容视为不可信 data，不赋予 instruction 权限。检索/网页内容要带来源边界；高风险 tool 调用由 policy engine 校验，敏感动作需要用户意图确认。仅靠 system prompt 提醒‘不要被注入’不够。

深挖解析

- 区分 control plane instruction 与 data plane content。
- 对网页/文档中的 tool-like 指令做隔离。
- 高权限 Agent 必须最小化可访问工具。

连续追问

Q1 Indirect prompt injection 怎么测？

Q2 网页中确实包含有用操作步骤怎么办？

易错回答 在 system prompt 再加一句‘忽略恶意指令’。

Know-Why Prompt injection 的根因是模型同时处理指令和数据，而传统文本边界并非真正安全边界。

Know-How 外部内容包装为 quoted evidence；Tool Gateway 按 user intent + policy 验证 action，不接受网页直接授权。

一句话收束 安全约束必须在模型无法绕过的执行层落地。

82. 什么叫 Least Privilege Agent?

频率 必考

难度 中

能力轴 Security、Permission

面试官在考什么 LLM 会犯错或被注入，降低权限就是降低错误的最大影响范围。

30 秒回答

Agent 在当前任务、当前时间、当前租户只拿到完成目标所需的最小工具、数据范围和操作权限，并尽量使用短时 token。能力应按任务动态授予，而不是永久拥有所有权限。

深挖解析

- 权限粒度包括 tool、resource、action、scope、duration。
- read/write 分离。
- 子 Agent 权限不应默认继承父 Agent 全部权限。

连续追问

Q1 动态授权怎么实现？

Q2 工具太多时如何做 capability token？

易错回答 给 Agent 一个管理员 API key，然后在 prompt 里限制。

Know-Why LLM 会犯错或被注入，降低权限就是降低错误的最大影响范围。

Know-How orchestrator 按 task 申请 scoped token；Tool Gateway 校验 claims，任务结束即失效。

一句话收束 安全约束必须在模型无法绕过的执行层落地。

83. 什么操作应该 Human-in-the-Loop?

频率 必考

难度 中

能力轴 Security、Permission

面试官在考什么 HITL 的目标是把人力放在错误代价最高的决策点。

30 秒回答

通常依据 impact、irreversibility、confidence、authorization 和合规要求。高金额转账、删除、发送外部消息、发布、权限变更等高影响不可逆动作适合审批；低风险可逆动作可自动执行。

深挖解析

- HITL 不是所有工具都确认。
- 审批时要展示意图、参数、证据和影响。
- 批准后签发一次性 approval token。

连续追问

Q1 如何减少审批疲劳？

Q2 用户批准后参数被 Agent 改了怎么办？

易错回答 按工具名称硬编码 ‘这个工具必须人工’ 而不看参数和影响。

Know-Why HITL 的目标是把人力放在错误代价最高的决策点。

Know-How 建立 risk scoring；超过阈值生成 immutable action preview，批准 token 绑定 action hash。

一句话收束 安全约束必须在模型无法绕过的执行层落地。

84. 执行代码的 Agent 为什么要 Sandbox?

频率 高频

难度 难

能力轴 Security、Permission

面试官在考什么 Agent 生成的代码不可完全信任，必须假设它可能意外或恶意访问系统资源。

30 秒回答

代码执行能力等于高权限任意操作入口。Sandbox 要限制 filesystem、network、process、CPU/memory/time、secret 和系统调用，最好使用短生命周期隔离环境，并只挂载必要工作目录。

深挖解析

- sandbox 也要考虑数据 exfiltration。
- 网络默认 deny，再按域名白名单。
- 产物通过受控 artifact channel 输出。

连续追问

Q1 Docker 就等于安全 sandbox 吗？

Q2 如何限制 shell command？

易错回答 只做一个工作目录路径检查就认为足够。

Know-Why Agent 生成的代码不可完全信任，必须假设它可能意外或恶意访问系统资源。

Know-How 使用容器/微虚拟机+seccomp/cgroup/network policy；凭证不写进环境，按任务临时注入最小权限。

一句话收束 安全约束必须在模型无法绕过的执行层落地。

85. Tracing 很重要，但 Trace 中包含用户敏感信息怎么办？

频率 高频

难度 中

能力轴 Security、Permission

面试官在考什么 可观测性和隐私是双重约束，不能为了 debug 创造新的数据泄漏面。

30 秒回答

Observability 也必须遵守最小化原则：敏感字段分类、默认脱敏、secret 永不记录、按风险采样、加密存储、短 TTL 和访问审计。调试需要原文时用受控、临时授权。

深挖解析

- trace 本身是敏感数据资产。
- prompt/tool input 不一定要全量保存。
- 跨租户日志必须隔离。

连续追问

Q1 如何既能复盘又不存原文？

Q2 PII 自动脱敏误删怎么办？

易错回答 为了可调试把所有 prompt、headers、API key 全存下来。

Know-Why 可观测性和隐私是双重约束，不能为了 debug 创造新的数据泄漏面。

Know-How 日志 SDK 在采集前做 field-level redaction；保存 hash/summary/metadata，原始内容按 break-glass 流程访问。

一句话收束 安全约束必须在模型无法绕过的执行层落地。

86. Multi-Tenant Agent 如何保证 A 公司数据不会进入 B 公司 Context?

频率 必考

难度 难

能力轴 Security、Permission

面试官在考什么 Agent 会跨很多组件流转数据，任何一层遗漏租户边界都可能串数据。

30 秒回答

从身份、检索、存储、缓存、tool token、trace 全链路携带可信 tenant_id，并在每个数据访问层强制隔离；不能依靠应用层传入的自然语言租户名。

深挖解析

- 向量检索必须 tenant filter。
- 缓存 key 必须包含 tenant/security context。
- 异步消息也要携带 signed identity。

连续追问

Q1 共享基础知识库怎么办？

Q2 tenant_id 被伪造怎么办？

易错回答 只在 system prompt 写 ‘你属于公司 A’ 。

Know-Why Agent 会跨很多组件流转数据，任何一层遗漏租户边界都可能串数据。

Know-How identity 从认证层签发并不可由模型修改；每个 storage/tool query 由服务端注入 scope。

一句话收束 安全约束必须在模型无法绕过的执行层落地。

87. 工具标记为 read-only、destructive、idempotent 有什么意义?

频率 高频

难度 中

能力轴 Security、Permission

面试官在考什么 把工具语义机器化后，才能把可靠性策略从 prompt 转成可执行规则。

30 秒回答

这些 metadata 让 runtime 能自动决定审批、重试、并行、缓存和风险策略。例如 destructive+non-idempotent 工具应避免自动 retry，read-only 工具更适合并行。

深挖解析

- Tool metadata 是 policy 输入。
- 描述性标签不能替代真正的权限校验。
- 还可加入 cost、latency、data sensitivity。

连续追问

Q1 标签由谁维护?

Q2 标签写错怎么办?

易错回答 只把标签展示给模型，不在 runtime 使用。

Know-Why 把工具语义机器化后，才能把可靠性策略从 prompt 转成可执行规则。

Know-How 注册工具时强制填写 side_effect/idempotent/risk/data_scope；Gateway 根据元数据生成默认 policy。

一句话收束 安全约束必须在模型无法绕过的执行层落地。

88. Agent 调第三方工具需要 Credential，Credential 应该放在哪里？

频率 必考

难度 中

能力轴 Security、Permission

面试官在考什么 模型上下文可能被记录、回显或受 prompt injection 影响，不是 secret storage。

30 秒回答

Credential 放在服务端 secret manager 或受控 broker，Agent context 里只放 capability reference；工具调用时由 Gateway 代注入短期凭据。避免把长期 API key 暴露给模型。

深挖解析

- 凭据要按 tenant/tool/scope 隔离。
- 短期 token 优于长期静态 key。
- trace 默认屏蔽 Authorization。

连续追问

Q1 用户自己的 OAuth token 怎么处理？

Q2 MCP server 需要凭据怎么办？

易错回答 把 API key 写进 system prompt 或 tool description。

Know-Why 模型上下文可能被记录、回显或受 prompt injection 影响，不是 secret storage。

Know-How 使用 secret broker；tool request 带 credential_handle，Gateway 在可信边界解析并注入。

一句话收束 安全约束必须在模型无法绕过的执行层落地。

89. 权限控制应该放在 Prompt、Agent、Tool Gateway 还是业务 API?

频率 必考

难度 难

能力轴 Security、Permission

面试官在考什么 任何单层都可能被绕过或出 bug；最终资源拥有者必须独立验证。

30 秒回答

越靠近资源越必须强制。Prompt 可用于引导，Agent runtime 可做早期拦截，Tool Gateway 做统一授权，业务 API 仍要做最终权限验证。安全关键约束不能只存在上层。

深挖解析

- Defense in depth。
- 业务 API 是最终 source of truth。
- 上层校验用于体验和减少无效调用，不取代后端授权。

连续追问

Q1 为什么要重复校验？

Q2 授权数据如何在多服务间传播？

易错回答 只选一个层做权限，其他层都信任。

Know-Why 任何单层都可能被绕过或出 bug；最终资源拥有者必须独立验证。

Know-How identity/claims 采用签名 token 透传；Gateway 与 API 都校验 scope/resource/action。

一句话收束 安全约束必须在模型无法绕过的执行层落地。

90. Memory Poisoning 怎么解决？

频率 必考

难度 难

能力轴 Security、Permission

面试官在考什么 一次错误记忆会跨多轮、跨任务持续影响决策，因此它是持久化攻击面。

30 秒回答

把记忆写入视为不可信更新：来源分级、写入验证、敏感/高影响事实需确认、记录 provenance 与 TTL；读取时按 trust/freshness 过滤。发现污染要支持追溯、撤销和批量重建。

深挖解析

- 外部网页内容不能直接变长期记忆。
- Agent 推断与用户明确事实要区分 trust level。
- procedural memory 更新尤其谨慎。

连续追问

Q1 怎么检测已经被污染的 memory？

Q2 用户故意给错误信息怎么办？

易错回答 把每轮模型总结直接写长期向量库。

Know-Why 一次错误记忆会跨多轮、跨任务持续影响决策，因此它是持久化攻击面。

Know-How memory record 携带 source/trust/verified_by；高影响写入进入 quarantine 或用户确认。

一句话收束 安全约束必须在模型无法绕过的执行层落地。

CHAPTER 10

性能、成本与综合系统设计

上线之后的 Agent 是成本、时延、可靠性和质量之间的多目标优化。

关键词 Latency / Cost / Scale / System Design

本章训练目标

- 能在白板上画出关键控制边界，而不是只报框架名。
- 能从 failure mode 反推防护机制、状态和指标。
- 面对连续追问时，始终回到“为什么 + 怎么落地”。

91. 一个 Agent 每次任务需要 200K tokens，怎么降到 50K?

频率 必考

难度 难

能力轴 性能、成本

面试官在考什么 成本优化的第一步是可归因；否则容易剪掉真正影响质量的上下文。

30 秒回答

先用 trace 找 token 去向，再分别优化：工具大结果外置、历史 compaction、检索 top-k/rerank、动态工具加载、prompt cache、小模型路由和结构化 state。不要一上来只改提示词。

深挖解析

- 按 model/tool/step 归因 token。
- 减少无关上下文通常比压缩输出收益大。
- 优化后必须检查 task success 是否退化。

连续追问

Q1 哪些 token 可以安全删？

Q2 Prompt cache 能省多少取决于什么？

易错回答 把 max_tokens 直接从 200K 改成 50K。

Know-Why 成本优化的第一步是可归因；否则容易剪掉真正影响质量的上下文。

Know-How 做 token flame graph；按最大来源逐项 A/B，记录 quality-per-dollar。

一句话收束 优化目标不是最低成本，而是单位成本下的可接受成功率与风险。

92. 是否所有步骤都需要最强模型？

频率 高频

难度 中

能力轴 性能、成本

面试官在考什么 Agent 是多步骤系统，各步骤对推理能力的需求不同，分层模型可以显著改善成本/延迟。

30 秒回答

不需要。分类、格式化、简单检索查询、摘要可用小模型；复杂规划、冲突裁决、关键推理使用强模型。路由依据任务复杂度、风险和置信度，失败可升级模型。

深挖解析

- 模型路由也是一种 fallback。
- 要避免小模型错误静默传播。
- 不同步骤使用不同模型版本要写入 trace。

连续追问

Q1 Router 怎么判断复杂度？

Q2 升级模型会不会让 latency 抖动？

易错回答 全流程只用最强模型，或为了省钱全部换小模型。

Know-Why Agent 是多步骤系统，各步骤对推理能力的需求不同，分层模型可以显著改善成本/延迟。

Know-How 离线统计各 step 类型在不同模型上的 success/cost，建立 policy + escalation threshold。

一句话收束 优化目标不是最低成本，而是单位成本下的可接受成功率与风险。

93. Agent 哪些东西适合缓存？

频率 高频

难度 中

能力轴 性能、成本

面试官在考什么 缓存能降低成本和延迟，也可能制造陈旧、越权和错误复用。

30 秒回答

稳定的 system/tool prefix、embedding、低时效 retrieval、幂等只读 tool result 和可复用中间 artifact 都可缓存；但缓存 key 必须包含版本、权限、租户与 freshness，副作用结果不能简单复用。

深挖解析

- cache invalidation 是核心。
- 敏感结果不能跨用户共享。
- prompt cache 与业务 cache 是不同层。

连续追问

Q1 RAG cache 如何避免旧知识？

Q2 模型输出能不能缓存？

易错回答 只按 query 文本做全局缓存。

Know-Why 缓存能降低成本和延迟，也可能制造陈旧、越权和错误复用。

Know-How 定义 cache key = input hash + version + tenant + policy context；每类缓存独立 TTL/invalidator。

一句话收束 优化目标不是最低成本，而是单位成本下的可接受成功率与风险。

94. 并行 10 个 Agent 一定比串行更快吗？

频率 高频

难度 中

能力轴 性能、成本

面试官在考什么 并行优化不是越多越好，而是受 Amdahl 定律和下游瓶颈限制。

30 秒回答

不一定。并行只缩短可并行关键路径，但会增加 API 限流、资源竞争、重复工作、上下文合并和尾延迟。应按 DAG、下游容量和任务粒度设 bounded concurrency。

深挖解析

- 关键看 critical path。
- 过细 task 会被调度开销吞噬。
- 并行成本要计入 token 和 coordination。

连续追问

Q1 怎么选择 worker 数？

Q2 什么时候 speculative execution 值得？

易错回答 把并发数等同于性能。

Know-Why 并行优化不是越多越好，而是受 Amdahl 定律和下游瓶颈限制。

Know-How 压测不同 concurrency，观察 throughput/P95/error/cost，找 sweet spot 并设置动态限流。

一句话收束 优化目标不是最低成本，而是单位成本下的可接受成功率与风险。

95. 怎么设计 Agent latency budget?

频率 必考

难度 难

能力轴 性能、成本

面试官在考什么 没有全局预算，局部优化无法保证 P95 SLO。

30 秒回答

先从用户 SLO 倒推，把预算分给 routing/planning、retrieval、tools、generation 和 reserve；按关键路径而非平均值分配。每个子调用接收 remaining deadline，必要时提前降级。

深挖解析

- 保留 retry/reserve budget。
- 并行阶段预算取 max 而非 sum。
- 尾延迟决定用户体验。

连续追问

Q1 模型生成时间波动大怎么预算？

Q2 超预算时先砍哪一步？

易错回答 所有组件各自追求最快，却没有 end-to-end deadline。

Know-Why 没有全局预算，局部优化无法保证 P95 SLO。

Know-How 从 trace 获取各 span P50/P95；按 critical path 设置 soft/hard deadline，runtime 动态降级。

一句话收束 优化目标不是最低成本，而是单位成本下的可接受成功率与风险。

96. Agent 怎么做 Load Test?

频率 高频

难度 难

能力轴 性能、成本

面试官在考什么 Agent 是复合工作负载，真实瓶颈常在工具、队列和长尾，而不是入口 QPS。

30 秒回答

不能只压 QPS，要模拟真实 session 长度、token 分布、工具延迟/失败、并发长任务、重试和模型 rate limit。测吞吐、queue age、P95/P99、failure、cost 和恢复行为。

深挖解析

- 负载模型应来自生产 trace 分布。
- 注入下游故障做 chaos test。
- 长连接/异步任务与普通 HTTP 压测不同。

连续追问

Q1 怎么测试 100 万日请求？

Q2 第三方 API 不允许压测怎么办？

易错回答 用固定 prompt 循环请求模型。

Know-Why Agent 是复合工作负载，真实瓶颈常在工具、队列和长尾，而不是入口 QPS。

Know-How 构建 workload generator + mocked downstream；回放匿名化 production traces，并做 failure injection。

一句话收束 优化目标不是最低成本，而是单位成本下的可接受成功率与风险。

97. 遇到 LLM Rate Limit 怎么办？

频率 高频

难度 中

能力轴 性能、成本

面试官在考什么 Agent 会生成突发多轮模型调用，rate limit 比单轮聊天更容易被放大。

30 秒回答

使用 client-side quota、队列、exponential backoff+jitter、模型/区域 fallback 和优先级；根据 deadline 决定等待还是降级。避免大量 Agent 同时重试形成 thundering herd。

深挖解析

- 尊重 retry-after。
- 按 tenant 做公平配额。
- 提前做 capacity planning。

连续追问

Q1 多个模型 provider 怎么路由？

Q2 fallback 模型能力不同如何保证结果？

易错回答 收到 429 所有请求立刻重试。

Know-Why Agent 会生成突发多轮模型调用，rate limit 比单轮聊天更容易被放大。

Know-How 中央 model gateway 做 token-bucket、queue、retry scheduling；provider error 触发 circuit breaker。

一句话收束 优化目标不是最低成本，而是单位成本下的可接受成功率与风险。

98. 100 个用户和 100 万用户的 Agent architecture 最大区别在哪里？

频率 必考

难度 难

能力轴 性能、成本

面试官在考什么 规模放大后，最难的是状态、一致性、尾延迟和租户隔离，而不是 LLM API 调用本身。

30 秒回答

从进程内 Agent loop 升级为分布式 runtime：无状态 worker、持久队列、durable state、租户隔离、全链路 trace、限流、容量规划、灰度发布和故障恢复都成为必需。

深挖解析

- 控制面和执行面分离。
- 长任务需要 scheduler/worker fleet。
- 模型和工具 gateway 平台化。

连续追问

Q1 数据库怎么分片？

Q2 如何做到跨区域容灾？

易错回答 只是把服务器从 1 台加到 100 台。

Know-Why 规模放大后，最难的是状态、一致性、尾延迟和租户隔离，而不是 LLM API 调用本身。

Know-How 采用 API -> scheduler -> durable queue -> workers -> gateways -> state/artifact stores；所有组件水平扩展。

一句话收束 优化目标不是最低成本，而是单位成本下的可接受成功率与风险。

99. Agent 成本怎么归因?

频率 必考

难度 中

能力轴 性能、成本

面试官在考什么 Agent 的成本由轨迹决定，同一用户请求可能触发数量差异很大的内部调用。

30 秒回答

至少能分解到 tenant/task/run/agent/model/tool/step，记录 input/output tokens、模型单价、工具费用、重试和缓存命中。只有成本可归因，才能知道优化哪个组件最值。

深挖解析

- 把 retry cost 单独统计。
- 关联 task success 计算 cost per successful task。
- 对多 Agent 汇总 parent-child cost。

连续追问

Q1 共享缓存成本怎么算？

Q2 如何做预算控制？

易错回答 只看整个月模型账单。

Know-Why Agent 的成本由轨迹决定，同一用户请求可能触发数量差异很大的内部调用。

Know-How 每个 span 计算 cost，聚合到 run；设置 per-task budget，超预算触发 smaller model/stop/HITL。

一句话收束 优化目标不是最低成本，而是单位成本下的可接受成功率与风险。

100. 综合系统设计：设计一个每天 100 万请求的企业级 Customer Support Agent。

频率 压轴

难度 难

能力轴 性能、成本

面试官在考什么 这道压轴题把 Agent 当作真实企业系统：面试官看的不是组件名，而是你能否让非确定决策在确定约束下安全运行。

30 秒回答

采用确定性入口与风险控制外壳：API Gateway -> Intent/Policy Router -> Orchestrator -> RAG/Tool/Multi-Agent Workers；所有外部动作经过 Tool Gateway，任务状态进入 Durable Store，长任务由 Queue+Worker 执行，HITL 用可恢复 checkpoint，Trace/Eval 贯穿全链路。

深挖解析

- P95<8s：短路径同步，长路径异步；并行 RAG 与只读工具，设置 deadline budget。
- 权限：tenant identity 贯穿检索、工具和日志；高风险动作 action hash+approval token。
- 可靠性：operation id、checkpoint、retry policy、circuit breaker、Saga；上下文用 artifact store+compaction。
- 质量：retrieval+trajectory+end-to-end eval；发布前 regression，线上 canary。

连续追问

- Q1 如何避免重复退款？
- Q2 Worker crash 如何恢复？
- Q3 Context 爆了怎么办？
- Q4 RAG 错误资料如何阻止错误动作？
- Q5 新模型上线如何证明更好？

易错回答 只画 LLM + Vector DB + Tools 三个框，忽略状态、权限、故障和评估。

Know-Why 这道压轴题把 Agent 当作真实企业系统：面试官看的不是组件名，而是你能否让非确定决策在确定约束下安全运行。

Q100 系统设计追问清单

白板顺序 先讲请求路径，再讲状态；随后画 Trust Boundary；最后用故障场景验证恢复、观测和预算。不要从组件名开始堆框。

- 1. 为什么这里需要 Multi-Agent，而不是一个大 Agent？
- 2. Orchestrator 如何判断子任务真正完成？
- 3. Agent 间消息丢失、重复、乱序分别怎么处理？
- 4. Worker crash 后如何从 checkpoint 恢复？
- 5. Tool timeout 时为什么不能直接 retry？
- 6. 如何避免重复退款或重复创建工单？
- 7. RAG 返回过期/错误资料时如何阻断错误动作？
- 8. ACL 应该在检索前还是检索后做？
- 9. Context Window 爆掉如何 compaction/reset？
- 10. Memory 冲突和过时如何处理？
- 11. Prompt Injection 如何防止触发高权限工具？
- 12. 高风险动作的 HITL 如何暂停与恢复？
- 13. Tool Gateway 应承担哪些可信控制？
- 14. 整个 8 秒 P95 的 latency budget 怎么分？
- 15. 下游 API 连续失败为什么需要 Circuit Breaker？
- 16. 任务执行到一半如何 cancel 并保证不留下幽灵操作？
- 17. Trace 需要记录哪些 span 才能做 failure attribution？
- 18. 新 Prompt/模型上线怎样做 regression + canary？
- 19. 如何把成本归因到 tenant/task/agent/step？
- 20. 成功率、成本、时延、安全如何放进同一个 scorecard？

Know-How 白板按 ①请求路径 ②状态流 ③权限边界 ④故障恢复 ⑤观测评估 ⑥成本/时延 六条线讲，最后用 3 个故障场景验证设计。

一句话收束 优化目标不是最低成本，而是单位成本下的可接受成功率与风险。

最后冲刺 | 20 道必刷题

- 02 不使用任何 Agent 框架，如何从零实现一个最小 Agent Loop?
- 04 生产 Agent 中，哪些逻辑交给 LLM，哪些逻辑必须写死在代码里?
- 10 请构建一个 Agent Failure Taxonomy。
- 12 Planner 输出的计划为什么不能直接执行?
- 13 Agent 执行一半跑偏了，如何在中间发现?
- 15 ReAct Agent 为什么经常死循环? 怎么识别?
- 21 Function Calling 背后模型如何决定调用哪个工具?
- 23 工具调用 timeout 了，你会直接 retry 吗?
- 24 支付 API 成功但响应丢失，Agent 认为失败并重试，如何避免重复执行?
- 30 设计一个 Production Tool Gateway。
- 33 多个 Agent 使用异步 JSON 消息通信，你会设计哪些字段?
- 34 Agent 消息丢失、重复、乱序分别怎么办?
- 35 主 Agent 怎么知道子 Agent 真正完成?
- 38 多个 Agent 相互调用进入无限‘甩锅循环’，怎么发现和阻断?
- 41 长任务 Context Window 快爆了怎么办?
- 43 摘要会丢关键细节，如何保证压缩后 Agent 不失忆?
- 61 一个 Agent 运行 40 分钟，第 39 分钟进程 crash，怎么办?
- 63 Tool 已执行成功，但 Agent 在写 checkpoint 前 crash，会发生什么?
- 71 Agent 每次 Run 到底应该 Trace 什么?
- 79 用户说‘Agent 答错了’，怎么判断是 Model、RAG、Tool 还是 Planner 的问题?

建议 如果只剩 2-3 天，不要再增加题量。把这 20 题练到能在白板上讲出 failure mode、控制点、状态、指标和恢复路径。

面试白板模板 | 一张图回答系统设计

1. Request Path 用户请求如何经过 Router / Orchestrator / Worker / Tool。

2. State Path Run State、Checkpoint、Artifact、Memory 放在哪里。

3. Trust Boundary 身份、ACL、Tool Gateway、HITL、Sandbox 在哪里。

4. Failure Path Timeout、Retry、Idempotency、Circuit Breaker、Saga 怎么走。

5. Observe Path Trace、Metrics、Eval、Failure Attribution 如何闭环。

6. Budget Path Token、Latency、Concurrency、Cost 如何受预算约束。

面试官最想听到的工程句式

- “我不会先直接 retry，我先判断这个操作是否幂等，以及 timeout 是否意味着结果未知。”
- “模型可以建议 action，但权限和业务 invariant 必须由可信执行层裁决。”
- “我会先找 first bad transition，而不是看到最终错误就直接改 prompt。”
- “Context 不等于状态；可压缩叙事和不可丢失业务状态必须分开。”
- “多 Agent 的本质问题是通信、状态、一致性和故障域，而不是角色扮演。”
- “我会把 success、latency、cost、unsafe action 放在同一个 scorecard，而不是只看回答准确率。”

7 天冲刺计划

Day 1 Q01-Q20: Loop + Planning。要求能手写最小 Agent Loop，并回答跑偏、循环、澄清、取消。

Day 2 Q21-Q40: Tool + Multi-Agent。重点练幂等、消息语义、Handoff、循环调用。

Day 3 Q41-Q60: Context + RAG。重点练 Compaction、Memory 冲突、ACL、RAG 评估。

Day 4 Q61-Q80: Runtime + Eval。重点练 crash consistency、Saga、Trace、Failure Attribution。

Day 5 Q81-Q99: Security + Performance。重点练 Prompt Injection、Least Privilege、成本与规模。

Day 6 Q100: 完整画一次百万请求 Customer Support Agent，连续回答 20 个追问。

Day 7 只刷 20 道必考题；每题先 30 秒，再 3 分钟，再接受 3 个追问。

术语速查

术语	面试表达
Agent Loop	模型—工具—观察—再决策的循环控制结构。
Harness	围绕模型提供工具、状态、上下文、权限、评估与恢复的运行环境。
Idempotency	同一操作重复执行不会造成额外业务副作用。
Checkpoint	可用于恢复执行的持久状态快照。
Compaction	压缩历史上下文以释放 token，同时保持任务连续性。
Context Reset	新建上下文，通过结构化 handoff 接续已持久化任务状态。
Handoff	Agent/阶段之间传递最小充分任务信息。
Saga	跨系统长事务通过补偿动作实现业务回滚的模式。
Circuit Breaker	下游持续失败时暂时停止调用，避免故障放大。
Failure Attribution	定位最早错误组件/步骤并解释失败传播路径。
Blast Radius	单次错误能够影响的最大资源、用户或后续步骤范围。
Trajectory Eval	对 Agent 中间步骤、工具选择和状态迁移进行评估。

参考资料与来源说明

S1 Micheal Lanham, AI Agents in Action, Manning, 2025.

用于 Agent / Multi-Agent / Actions & Tools / Memory & Knowledge / Reasoning & Evaluation / Planning & Feedback 的知识结构参考。

S2 凌峰, 《AI Agent 开发与应用: 基于大模型的智能体构建》, 清华大学出版社, 2025。

用于感知-决策-执行、上下文与记忆、ReAct、动态工具集成、多智能体通信与容错等结构参考。

S3 《AI Agent 智能工作流》。

用于 Agent 工具调用、MCP、记忆压缩、企业安全架构、监控与沙箱等主题参考。

S4 《字节跳动 RAG 实践手册》。

用于索引、检索触发、Hybrid/Rerank、效果评估、全链路监控、故障复盘、成本归因与权限安全等 RAG 主题参考。

重要说明

本手册不是上述资料的摘要或替代品。题目、答案和 Production Engineering 扩展均为面试训练目的重新组织。尤其是幂等、Saga、Circuit Breaker、deadline、版本迁移等分布式系统机制, 是为了把 Agent 问题提升到“可上线、可恢复、可治理”的系统工程语境。

让 Agent 成功并不难; 真正的工程能力, 是让它可控地失败。

— END —